

Лабораторная работа №1. Знакомство со средой Unity

Цель работы

Целью работы является знакомство со средой разработки игровых приложений Unity 3D, создание первого проекта и знакомство со средствами управления, а также интерфейсом Unity.

1. Основы работы в среде Unity

1.1. Достоинства Unity

Unity — профессиональный игровой движок, который используется при создании видеоигр для различных платформ. Это достаточно прямой ответ на прямой вопрос «Что такое Unity?». Но что конкретно он означает? И чем примечателен инструмент Unity?

1.1.1. Сильные стороны и преимущества Unity

Любой игровой движок предоставляет множество функциональных возможностей, которые задействуются в различных играх. Реализованная на этом движке игра получает все эти функциональные возможности, кроме того, добавляются ее собственные игровые ресурсы и код игрового сценария. Unity предлагает моделирование физических сред, карты нормалей, преграждение окружающего света в экранном пространстве (Screen Space Ambient Occlusion, SSAO), динамические тени... список можно продолжать долго. Подобным набором функциональных возможностей могут похвастаться многие игровые движки, но у Unity есть два основных преимущества перед другими передовыми инструментами разработки игр: чрезвычайно производительный визуальный рабочий процесс и мощная межплатформенная поддержка.

Визуальный рабочий процесс представляет собой достаточно уникальную вещь, выделяющую данный инструмент из большинства других сред разработки игр. В то время как остальные инструменты разработки игр зачастую представляют собой мешанину разрозненных частей, которые требуется контролировать, или, возможно, библиотеку, для работы с которой нужно настраивать собственную интегрированную среду разработки (Integrated Development Environment, IDE), цепочку сборки и прочее в этом роде, рабочий процесс в Unity привязан к тщательно продуманному визуальному редактору. В этом редакторе вы будете компоновать сцены будущей игры, связывая игровые ресурсы и код в интерактивные объекты. Именно он позволяет быстро и рационально создавать профессиональные игры, обеспечивая невиданную продуктивность труда разработчиков и предоставляя в их распоряжение исчерпывающий перечень самых современных технологий в области видеоигр.

Особенно полезен этот редактор при разработке проектов с последовательным улучшением, например в циклах создания прототипов или

тестирования. Корректировать объекты в редакторе и двигать элементы в сцене можно даже при запущенной игре. Кроме того, Unity позволяет настраивать и сам редактор при помощи сценариев, добавляющих новые функциональные особенности и элементы меню к интерфейсу. В дополнение к значительным преимуществам в плане производительности, которые нам дает редактор, у набора инструментов Unity существует еще и сильная межплатформенная поддержка. В данном случае под этим словосочетанием подразумеваются не только места развертывания (вы можете развернуть игру на персональном компьютере, в Интернете, на мобильном устройстве или на консоли), но и инструменты разработки (создание игры может осуществляться на машинах, работающих под управлением как Windows, так и Mac OS). Эта независимость от платформы явилась результатом того, что изначально инструмент Unity предназначался исключительно для компьютеров Mac, а позднее был перенесен на машины с операционными системами семейства Windows.

Между тем в дополнение к этим основным достоинствам идет и третье, менее бросающееся в глаза преимущество, обеспечиваемое модульной системой компонентов, которая используется для конструирования игровых объектов. «Компоненты» в такой системе представляют собой комбинируемые пакеты функциональных элементов, поэтому объекты создаются как наборы компонентов, а не как жесткая иерархия классов. Другими словами, компонентная система являет собой альтернативный (и обычно более гибкий) подход к объектно-ориентированному программированию, в котором игровые объекты создаются путем объединения, а не наследования. Сравнение подходов демонстрирует диаграмма на рис. 1.1.



Рис. 1.1. Сравнение наследования и компонентной системы

В компонентной системе объект существует в горизонтальной иерархии, поэтому различные объекты состоят из разных наборов компонентов, а не из

структуры наследования, в которой разные объекты оказываются на разных ветвях дерева. Такая компоновка облегчает создание прототипов, потому что взять нужный набор компонентов куда быстрее и проще, чем перестраивать цепочку наследования при изменении каждого объекта.

Разумеется, ничто не мешает вам написать код, реализующий вашу собственную компонентную систему, но в Unity уже существует вполне надежный вариант такой системы, органично встроенный в визуальный редактор. Поэтому у вас есть возможность не только программно управлять компонентами, но и устанавливать и разрывать связи между ними в редакторе. Разумеется, ваши возможности не ограничиваются составлением объектов из готовых деталей; в своем коде вы можете воспользоваться наследованием и всеми наработанными на его базе паттернами проектирования.

1.1.2. Недостатки, о которых нужно знать

Инструмент Unity имеет множество достоинств, которые превращают его в замечательное средство разработки игр, но с моей стороны было бы упущением не упомянуть о его недостатках. В частности, сочетание визуального редактора со сложным кодом при всей его эффективности в рамках компонентной системы Unity является нетипичным и может вызвать затруднения. В сложных сценах можно потерять из виду некоторые из присоединенных компонентов. В Unity существует функция поиска, позволяющая обнаруживать присоединенные сценарии, но она могла бы быть и более надежной — порой возникают ситуации, когда для поиска связанных сценариев приходится вручную просматривать все элементы сцены. Такое случается не часто, тем не менее этой кропотливой и трудоемкой работы хотелось бы совсем избежать.

Другим неожиданным и обескураживающим для опытных программистов недостатком является тот факт, что Unity не поддерживает ссылки на внешние библиотеки кода. Все доступные библиотеки, которые вы планируете задействовать, следует вручную скопировать в проект, вместо того чтобы просто дать ссылку на одну папку общего доступа. Отсутствие единой папки с библиотеками затрудняет коллективное использование функционала разными проектами. Это неудобство можно обойти, рационально применяя системы контроля версий, но готовое решение данной проблемы в Unity отсутствует.

Третий недостаток связан с использованием шаблонов экземпляров (prefabs). Шаблоны экземпляров предлагают гибкий подход к визуальному созданию интерактивных объектов. Эта крайне мощная концепция существует исключительно в Unity (и, естественно, она связана с компонентной системой Unity), но редактирование таких шаблонов порой оказывается на удивление труднореализуемым. Я считаю шаблоны экземпляров полезным и важным аспектом работы с Unity и надеюсь, что в будущих версиях способ их редактирования будет усовершенствован.

1.2. Визуальный интерфейс Unity

В предыдущем разделе мы много говорили о том, как выгоден встроенный в Unity визуальный редактор с точки зрения производительности, а сейчас пришло время познакомиться с его интерфейсом и узнать, как он работает. Если вы пока этого не сделали, скачайте программу со страницы <http://unity3d.com/ru/get-unity> и установите на свой компьютер (обязательно установите флажок Example Project, если установщик его сбросит). После этого запустите Unity, чтобы приступить к исследованию интерфейса.

Современные версии Unity поставляются совместно с вспомогательным приложением Unity Hub, которое представляет собой удобное средство для обзора ваших проектов, а также включает возможность просмотра обучающих проектов и управления различными версиями Unity. Впрочем для работы оно не является обязательным (рис. 1.2).

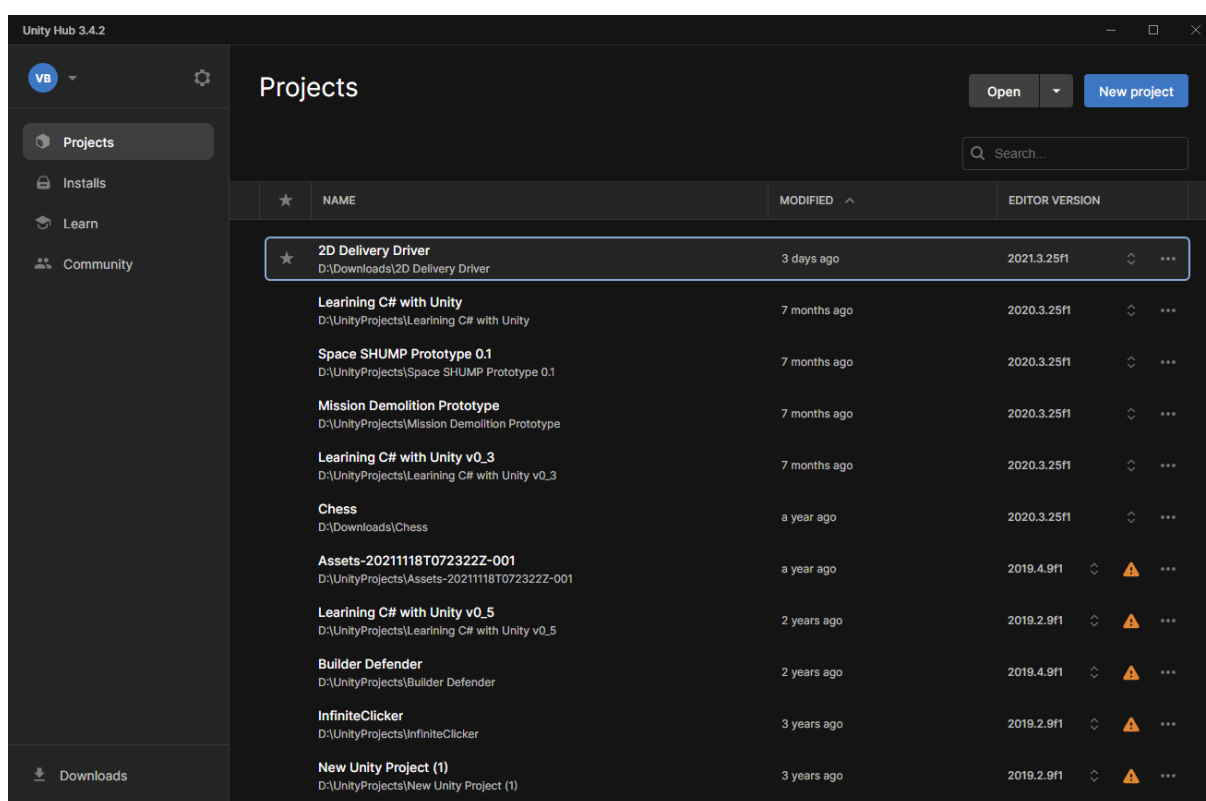


Рис. 1.2. Интерфейс Unity Hub

Для создания нового проекта необходимо нажать кнопку New, выбрать тип проекта, указать его название и местоположение (рис. 1.3).

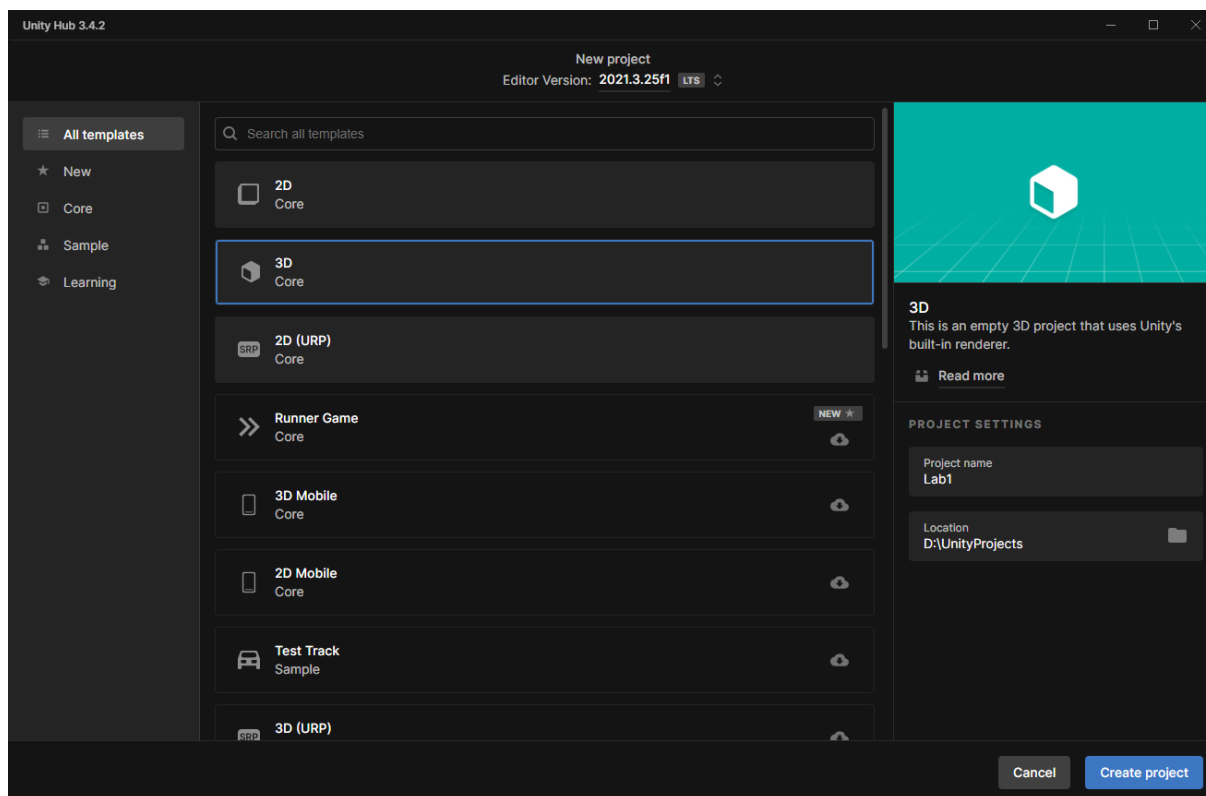


Рис. 1.3. Создание нового проекта

Вам, скорее всего, потребуется пример, поэтому откройте соответствующий проект; при установке новой копии этот проект должен предлагаться автоматически, но вы можете выбрать в меню File команду Open Project и открыть его вручную. Он находится в пользовательской папке общего доступа, адрес которой в операционных системах семейства Windows выглядит примерно так: C:\Users\Public\Documents\Unity Projects\

Если заодно потребуется открыть пример сцены, дважды щелкните на файле Car (рис. 1.4 демонстрирует, что такие файлы в Unity обозначаются символом куба). Значок этого файла в средстве просмотра файлов, расположенном в нижней части редактора, находится по адресу SampleScenes/Scenes/. Вы должны получить экран, показанный на рис. 1.4.

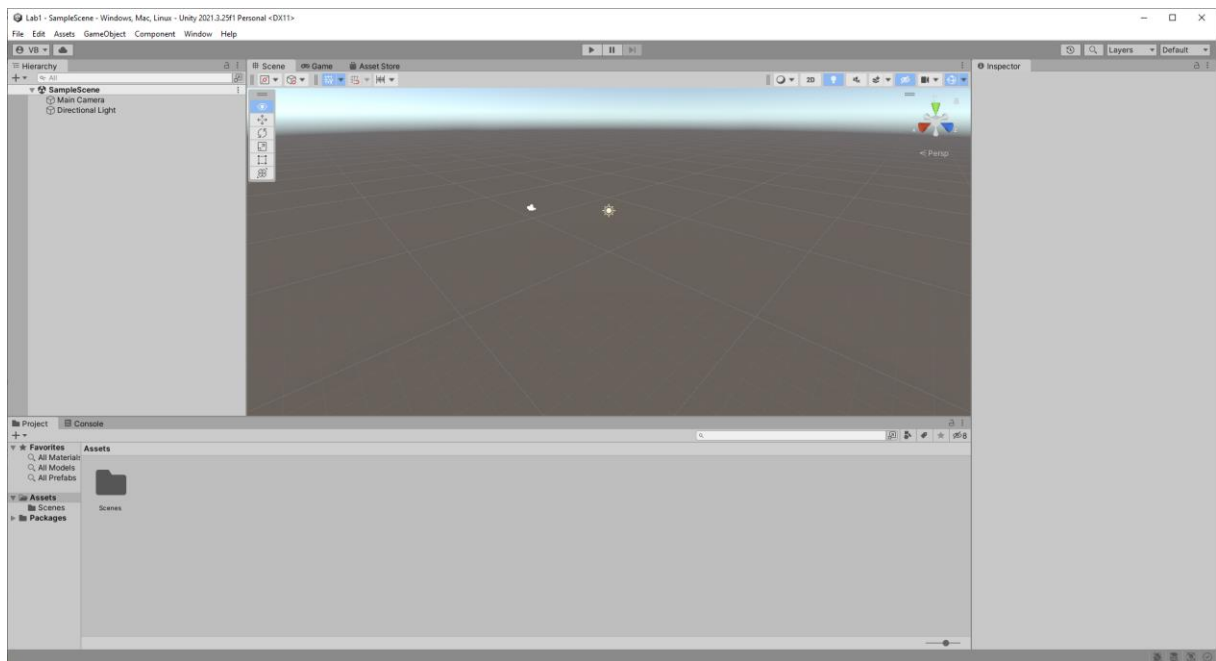


Рис. 1.4. Интерфейс среды Unity

Интерфейс Unity разбит на несколько частей: вкладка Scene, вкладка Game, панель инструментов, вкладка Hierarchy, панель Inspector, вкладки Project и Console. У каждой части есть собственное предназначение, при этом все они играют важную роль в цикле создания игры:

- Просмотр файлов выполняется на вкладке **Project**.
- Помещенные в трехмерную сцену объекты просматриваются на вкладке **Scene**.
- Панель инструментов предоставляет вам элементы управления сценой.
- Менять взаимосвязи между объектами можно методом перетаскивания на вкладке **Hierarchy**.
- Панель **Inspector** отображает информацию о выделенных объектах, в том числе и о связанном с ними коде.
- Тестировать полученные результаты можно на вкладке **Game**, одновременно просматривая сообщения об ошибках на вкладке **Console**.

Эта компоновка предлагается по умолчанию; все доступные представления помещены на вкладки, которые можно перемещать, можно менять их размер и фиксировать в разных частях экрана. Для удобства предлагается расположить панели в соответствии с шаблоном, представленным на рис. 1.5. Для этого нужно передвинуть панель Game в нижнюю часть экрана, а панель проекта в левую, и сделать расположение в одну колонку. Расположение панелей можно сохранить через меню Window -> Layouts.

При необходимости можно вернуть стандартное расположение панелей через меню Window -> Layouts -> Default.

Панель **Hierarchy** отображает текстовый список всех объектов сцены и их взаимосвязей. Для связывания объектов нужно перетащить их на данную панель

Верхнюю часть целиком занимает панель инструментов. Слева кнопки для осмотра и перемещения объектов, а в центре располагается кнопка **Play**

Вкладки **Scene** и **Game** предназначены для просмотра трехмерных сцен и воспроизведения игр соответственно

Вкладки **Project** и **Console** предназначены для просмотра всех файлов проекта и сообщений, касающихся кода программы, соответственно

С правой стороны располагается панель **Inspector**, отображающая информацию о выделенном в данный момент объекте

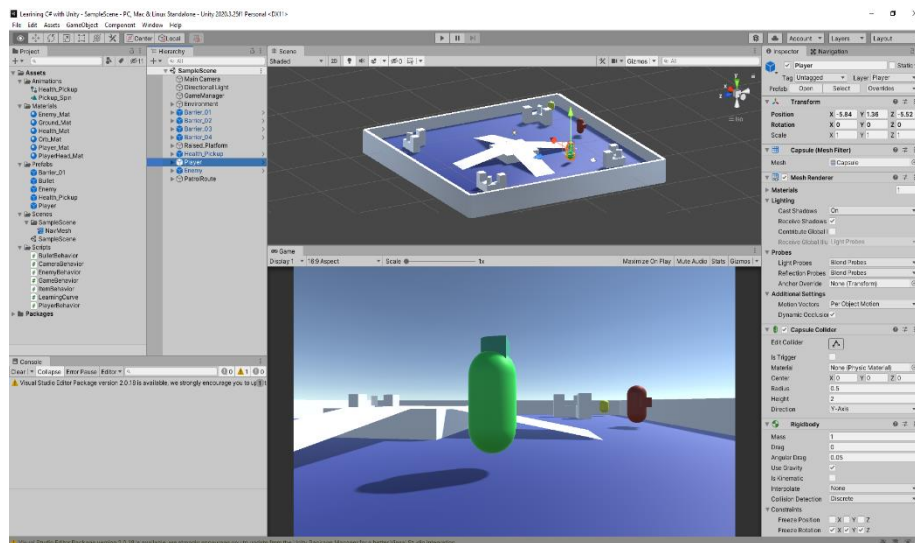


Рис. 1.5. Пример прототипа игры The Hero Born. Расположение панелей с отображением игрового окна и уменьшенным размером окна проекта

1.2.1. Вкладка **Scene**, вкладка **Game** и панель инструментов

Наиболее заметной частью интерфейса является расположенная в центре вкладка **Scene**. Именно здесь можно видеть, как выглядит мир игры, и перемещать объекты в сцене. Сеточные объекты в сцене выглядят, как им и положено, в виде сеток. Также можно наблюдать и ряд других объектов, представленных различными значками и цветными линиями. Это камеры, источники света, источники звука, области столкновений и т.д. Разумеется, наблюдаемая тут картинка отличается от того, что вы будете видеть в процессе игры, — можно рассматривать сцену, не ограничиваясь игровым представлением.

Сеточный объект (mesh object) — это визуализация объекта в трехмерном пространстве. Она создается из набора соединенных друг с другом линий и форм, которые формируют сетку.

Игровое представление отображается не на отдельной части экрана, а на вкладке **Game**, расположенной рядом с вкладкой **Scene** (переход с вкладки на вкладку осуществляется с помощью кнопок в верхнем левом углу области отображения). В интерфейсе есть и другие элементы, сконструированные подобным образом; для изменения отображаемого ими содержимого достаточно перейти на другую вкладку. После запуска игры начинает отображаться игровое представление, то есть вам не нужно переходить на вкладку **Game** — переключение выполняется автоматически.

Для запуска игры достаточно щелкнуть на кнопке **Play**, расположенной над вкладкой **Scene**. Вся верхняя часть интерфейса занята так называемой панелью инструментов и кнопка **Play** находится как раз в центре этой панели.

Рисунок 1.6 получен отбрасыванием остальной части интерфейса редактора и демонстрирует только панель инструментов с расположенными под ней вкладками Scene и Game.

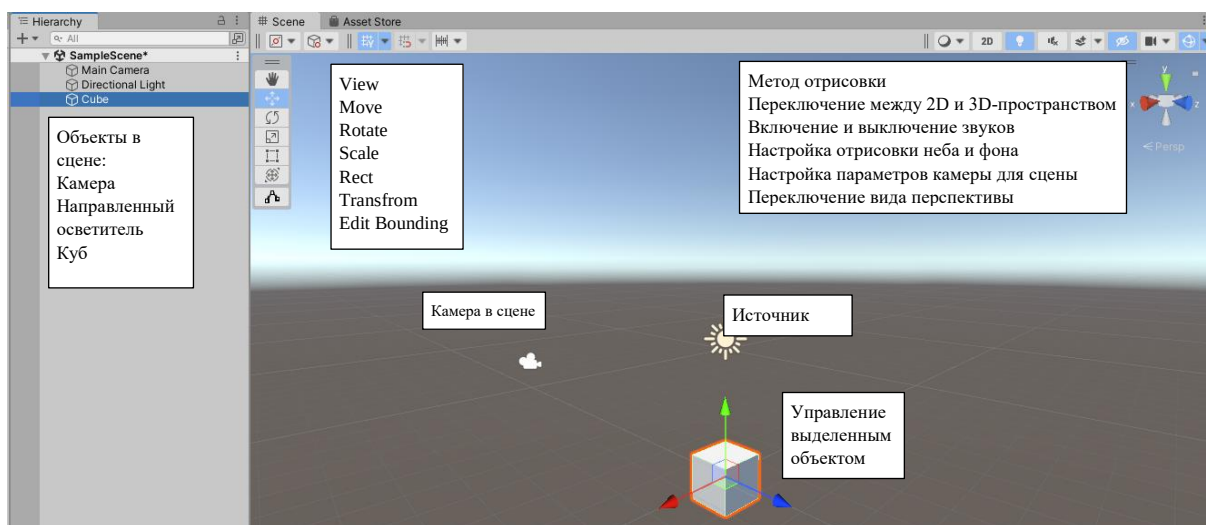


Рис. 1.6. Интерфейс работы с панелью сцены. Данный интерфейс позволяет изменять отображение объектов в сцене, настраивать их положение и другие параметры

На левой стороне панели инструментов расположены кнопки навигации и преобразования объектов. Они позволяют рассматривать сцену с разных сторон и перемещать объекты. Предлагаю вам попрактиковаться в их применении, так как просмотр сцен и перемещение объектов — это два основных действия, выполняемых в визуальном редакторе Unity. Правую сторону панели инструментов занимают раскрывающиеся меню с перечнем компоновок и слоев. Интерфейс Unity является гибкой, настраиваемой системой, поэтому и существует меню **Layouts**, позволяющее переходить от одного варианта компоновки к другому. Меню **Layers** относится к расширенным функциональным возможностям управления слоями, которое позволяет редактировать способы отображения объектов, а также настраивать их взаимодействие друг с другом.

1.2.2. Работа с мышью и клавиатурой

Навигация в сцене осуществляется в основном с помощью мыши и набора клавиш модификаторов, влияющих на результат манипуляций мышью. Три главными операциями являются перемещение (**move**), облёт (**orbit**) и масштабирование (**zoom**). В основном они сводятся к щелчкам и перетаскиванию при нажатых и удерживаемых клавишах Alt и Ctrl. Потратьте некоторое время на манипуляции объектами сцены, чтобы понять, как выполняются перемещение, облёт и масштабирование.

Преобразование объектов также осуществляется посредством этих трех операций. Более того, каждому типу навигации соответствует собственное

преобразование: перенос (**translate**), поворот (**rotate**) и изменение размеров (**scale**).

1.2.3. Вкладка Hierarchy и панель Inspector

На левой стороне экрана расположена вкладка **Hierarchy**, в то время как правую заняла панель **Inspector** (рис. 1.7). Вкладка Hierarchy содержит список всех присутствующих в сцене объектов в виде древовидной структуры, ветви которой вложены друг в друга в соответствии с иерархическими связями между объектами. По сути, эта вкладка предлагает вам возможность выделения объектов по именам, избавляя от необходимости искать нужный объект в сцене, чтобы выделить его щелчком. Иерархические связи объединяют объекты друг с другом. Визуально это оформлено в виде папок, при этом вы можете перемещать одновременно целые группы объектов.

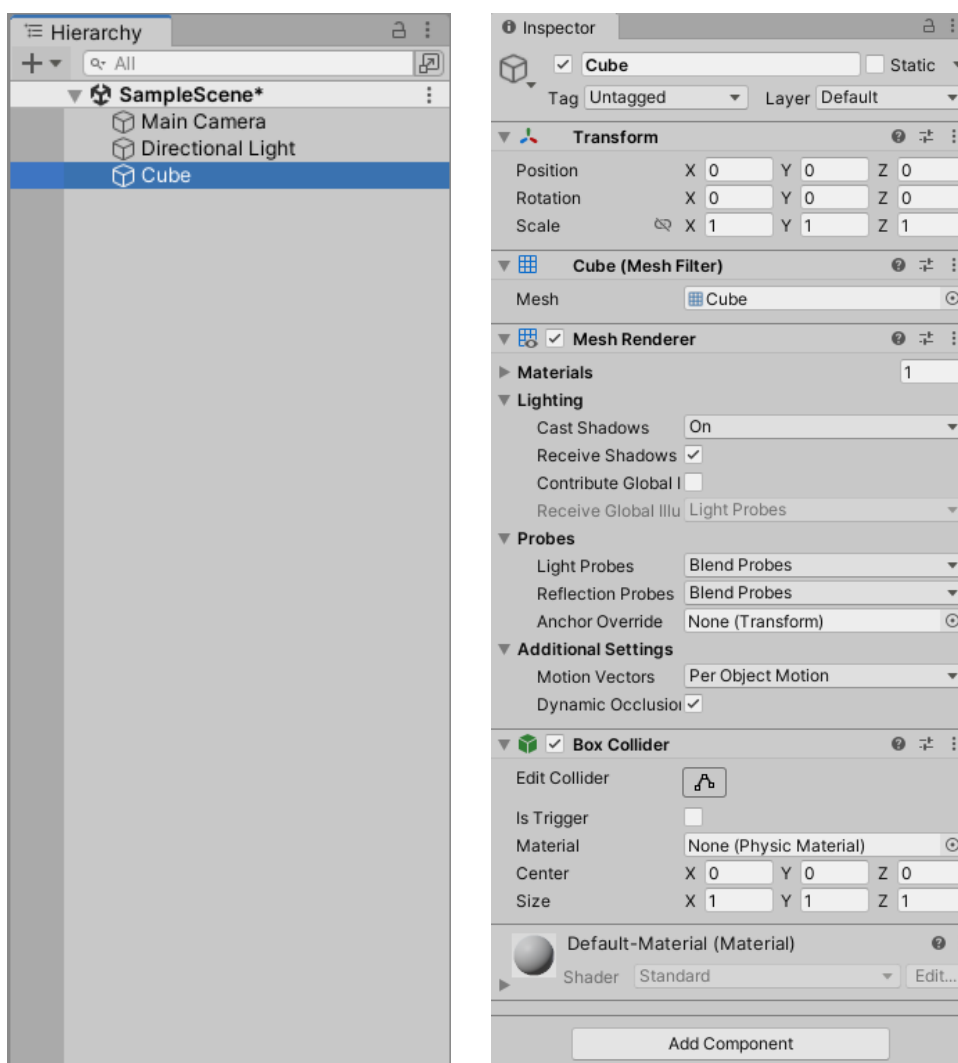


Рис. 1.7. Вкладка Hierarchy и панель Inspector

На панели Inspector отображаются данные выделенного в текущий момент объекта. Выделите любой объект, и вы немедленно увидите, как изменится вид панели Inspector. Отображаемая информация по большей части представляет

собой список компонентов, причем вы можете добавлять к объектам новые компоненты или удалять существующие. Все игровые объекты содержат по крайней мере один компонент — Transform, — поэтому на панели Inspector вы всегда будете видеть хотя бы сведения о положении и ориентации выделенного объекта. У многих объектов есть целые списки компонентов, в том числе связанные с этими объектами сценарии.

1.2.4. Вкладки Project и Console

В нижней части экрана находятся показанные на рис. 1.8 вкладки Project и Console. В данном случае мы видим пример такой же организации элементов интерфейса, как и у вкладок Scene и View, что легко позволяет переходить от одного представления к другому. На вкладке Project отображаются все ресурсы (графические фрагменты, код и т. п.) проекта. В левой части этой вкладки вы видите список папок проекта; при выделении папки справа появляются находящиеся в ней файлы. По сути, это такой же список, как и на вкладке Hierarchy, но если эта вкладка показывает вам перечень объектов сцены, то на вкладке Project представлены файлы, не включенные ни в одну конкретную сцену (в том числе и сами файлы сцен — сохраненная сцена появляется на вкладке Project!).



Рис. 1.8. Вкладки Project и Console

Вкладка Console представляет собой место, где выводятся сообщения, связанные с кодом. Иногда это намеренно вставленные вами в программу сообщения отладчика, иногда это сообщения Unity, появляющиеся при обнаружении ошибок в написанном вами сценарии.

1.3. Готовимся программировать в Unity

Теперь давайте рассмотрим, как происходит процесс программирования в Unity. Хотя графические объекты – **Ассеты** можно расположить в визуальном редакторе, необходимо написать код, чтобы управлять ими и сделать игру интерактивной. Сложное программирование в Unity осуществляется с использованием языка программирования C#.

Запустите Unity и создайте новый проект: выберите New в Unity Hub или File > New Project, если Unity уже запущен. Введите название проекта, оставьте по умолчанию шаблон 3D (в последующих главах будет упомянуто 2D), а затем выберите место, где вы хотите сохранить проект. Проект Unity – это просто каталог, наполненный различными файлами активов и настроек, поэтому

сохраните проект в любом месте на вашем компьютере. Нажмите кнопку Создать, после чего Unity ненадолго исчезнет, пока создается каталог проекта.

Выберите Add в Unity Hub, чтобы добавить папку загруженного проекта в список, а затем щелкните проект в списке.

ПРИМЕЧАНИЕ

Выполнение кода в Unity всегда начинается с файлов, связанных с объектом сцены. В конечном счете все это часть упомянутой ранее компонентной системы; игровые объекты строятся как наборы компонентов, и каждый такой набор может включать в себя исполняемый сценарий.

В терминологии Unity файлы с кодом принято называть сценариями, используя для определения этого слова значение, чаще всего применяемое к ситуации, когда в браузере запускается JavaScript: код выполняется внутри игрового движка Unity, а не фигурирует после компиляции в виде самостоятельного фрагмента. Но зачастую люди определяют данный термин совсем по-другому. К примеру, сценариями иногда называют автономные служебные программы. Сценарии в Unity больше напоминают индивидуальные классы ООП. Присоединенные к объектам сцены сценарии являются экземплярами объектов.

Как вы, наверное, догадались из этого описания, сценарии в Unity представляют собой компоненты. Но прошу заметить, не все сценарии, а только наследующие от класса MonoBehaviour, базового класса компонентов-сценариев. Этот класс определяет способ присоединения компонентов к игровым объектам (как показано в листинге 1.1). Наследование от данного класса дает пару автоматически добавляемых методов, которые вы можете переопределить. Это метод Start(), вызываемый при активации объекта (а она, как правило, наступает после загрузки содержащего объект уровня), и метод Update(), вызываемый в каждом кадре. Соответственно, ваш код запускается, когда вы помещаете его в эти предустановленные методы.

Листинг 1.1. Шаблон кода для базового компонента сценария

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class Test : MonoBehaviour
{
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```

```
}  
}
```

Именно так выглядит файл, когда вы создаете новый сценарий на C#: минимальный шаблонный код, определяющий корректный компонент в Unity. Существует шаблон сценария, скрытый в недрах Unity. При создании нового сценария приложение копирует этот шаблон и переименовывает класс в соответствии с именем файла (в моем случае — в Test.cs). Есть и пустые оболочки методов Start() и Update(), так как именно из них чаще всего вызывается создаваемый код.

Чтобы создать сценарий, выберите команду C# Script в меню Create, доступ к которому можно получить, открыв меню Assets (обратите внимание, что как в меню Assets, так и в меню GameObjects есть варианты команды Create, но это разные команды) или щелкнув правой кнопкой мыши в произвольной точке вкладки Project. Введите имя нового сценария, например «HelloWorld».

Для того, чтобы созданный сценарий выполнялся при запуске игры, необходимо прикрепить его к одному из объектов в сцене. Если этот сценарий по логике не принадлежит конкретному объекту, а служит для управления сценарием игры в целом, то его можно прикрепить к объекту Main Camera, либо к специально созданному пустому объекту (Create Empty). В дальнейшем для таких сценариев будет использоваться специально создаваемый объект GameManager.

1.3.1. Вывод на консоль: «Hello World!»

Итак, в нашем проекте появился пустой сценарий, но пока отсутствует объект, к которому этот сценарий можно было бы присоединить. Вспомните рис. 1.1, демонстрирующий работу системы компонентов; любой сценарий является компонентом, поэтому его нужно превратить в один из компонентов какого-то объекта. Выберите в меню Select GameObject команду Create Empty, и на вкладке Hierarchy появится пустой объект GameObject. Теперь перетащите значок сценария с вкладки Project на вкладку Hierarchy и поместите на строку GameObject. Как показано на рис. 1.15, приложение Unity выделяет места, куда можно поместить сценарий. Как только вы отпустите кнопку мыши, сценарий будет присоединен к объекту GameObject. Чтобы удостовериться в успешном исходе операции, выделите объект и посмотрите на панель Inspector. Вы должны увидеть там два компонента: Transform — базовый компонент положения/ориентации/масштаба, присутствующий у всех объектов, который невозможно удалить, а сразу под ним — ваш сценарий.

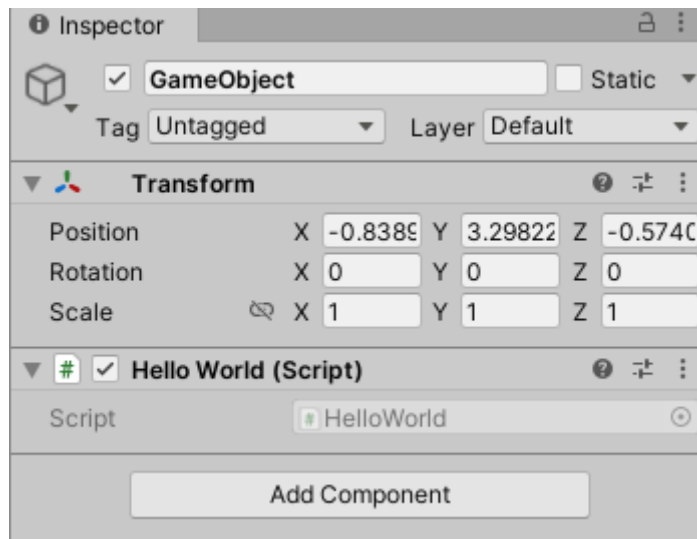


Рис. 1.9. Связанный сценарий отображается на панели Inspector

После связывания сценария с объектом на панели Inspector вы увидите примерно такую картину, которая показана на рис. 1.16. Сценарий будет фигурировать в качестве компонента. Теперь он начнет исполняться при воспроизведении сцены, хотя вы не заметите никаких внешних проявлений, так как код еще не написан. Откройте сценарий, чтобы вернуться к листингу 1.1. Классическим примером, с которого всегда начинается знакомство с новой средой программирования, становится вывод текста «Hello World!». Поэтому добавьте эту строку в метод `Start()`, как показано в листинге 1.2.

Листинг 1.2. Вывод сообщения в консоль

```
void Start()
{
    Debug.Log("Hello World!");
}
```

Команда `Debug.Log()` выводит сообщение на вкладку Console в Unity. Строка с этой командой идет в метод `Start()`, так как, как мы уже упоминали, данный метод вызывается однократно после активации объекта. Другими словами, после щелчка на кнопке Play в редакторе метод `Start()` будет вызван всего один раз. Как только вы добавили в сценарий команду регистрации (обязательно сохраните сценарий), щелкните на кнопке Play в программе Unity и перейдите на вкладку Console. Там появится сообщение «Hello World!».

1.4. Планирование проекта

Перед тем как приступить к программированию, всегда нужно остановиться и ответить на вопрос «что я собираюсь построить?». Проектирование игр — весьма обширная тема, которой посвящено множество толстых книг. К счастью, в данном случае для разработки базового учебного проекта достаточно мысленно представлять структуру будущей сцены. Первые

проекты будут несложными, чтобы лишние детали не мешали вам изучать принципы программирования; а задумываться о разработке более высокоуровневого проекта можно после того, как вы освоите основы создания игр.

Вашим первым проектом станет создание сцены из шутера от первого лица — мы будем обозначать этот тип игр аббревиатурой FPS (First-Person Shooter). Вы построите комнату, по которой можно перемещаться, при этом игроки будут наблюдать окружающий мир с точки зрения игрового персонажа. Управлять этим персонажем игрок сможет посредством мыши и клавиатуры. Все интересные, но сложные элементы готовой игры мы пока отбросим, чтобы сконцентрироваться на основной задаче — перемещениях в трехмерном пространстве. Рисунок 1.10 иллюстрирует сценарий проекта, по сути, представляющий собой придуманный мной перечень действий:

1. Разработка комнаты: создание пола, внешних и внутренних стен.
2. Размещение источников света и камеры.
3. Создание объекта-игрока (в том числе и присоединение камеры к его верхней части).
4. Написание сценариев перемещения: повороты при помощи мыши и перемещения при помощи клавиатуры.

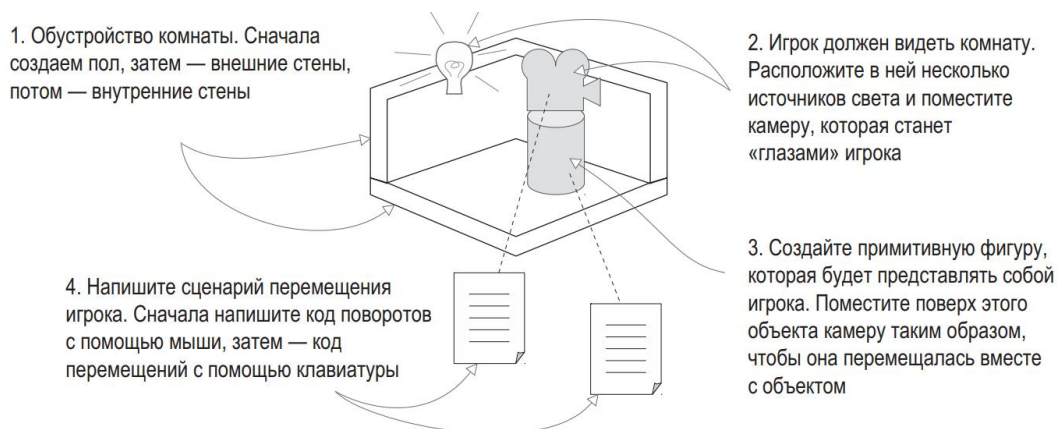


Рис. 1.10. Сценарий трехмерного демонстрационного проекта

Разделы, посвященные сценариям перемещения, так велики только потому, что мы подробно рассматриваем каждую деталь, чтобы полностью понять принципы программирования. Мы начинаем с игры от первого лица, чтобы снизить требования к художественному оформлению — в играх от первого лица себя игрок не видит, поэтому вполне допустимо придать персонажу форму цилиндра, на верхней грани которого закреплена камера. Теперь осталось понять, как функционируют трехмерные координаты, и вы сможете легко вставлять объекты в сцены в визуальном редакторе.

1.5. Трехмерное координатное пространство

Сформулированный нами для начала простой план включает в себя три аспекта: игровое пространство, средства наблюдения, элементы управления. Для их реализации требуется понимание того, каким образом в трехмерных симуляциях задаются положение и перемещение объектов. Те, кто раньше никогда не сталкивался с трехмерной графикой, вполне могут этого не знать. Все сводится к числам, указывающим положение точки в пространстве. Сопоставление этих чисел с пространством происходит через оси системы координат. На рис. 1.11а оси X и Y показаны двумерные координаты. Это случай, когда все точки лежат в одной плоскости. Трехмерное пространство задается уже тремя координатными осями. Так как ось X располагается на странице горизонтально, а ось Y — вертикально, третья ось должна как бы «протыкать» страницу, располагаясь перпендикулярно осям X и Y . Рисунок 1.11б демонстрирует оси X , Y и Z для трехмерного координатного пространства. У всех элементов, обладающих определенным положением в сцене (у игрока, у стен и т. п.), будут координаты XYZ .

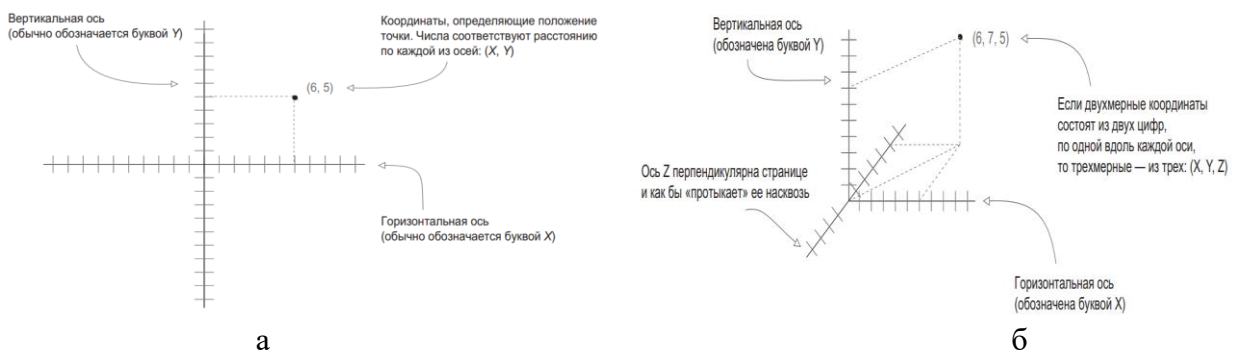


Рис. 1.11 а) Координаты по осям X и Y определяют положение точки на плоскости, б) Координаты по осям X , Y и Z определяют точку в трехмерном пространстве

На вкладке Scene в Unity вы видите значок трех осей, а на панели Inspector можно указать три числа, задающих положение объекта. Координаты в трехмерном пространстве вы будете использовать не только при написании кода, определяющего местоположение объектов, но и для указания смещений как значений сдвига вдоль каждой из осей.

Теперь, когда у вас есть не только план действий, но и представление о том, как с помощью координат задать положение объекта в трехмерном пространстве, приступим к работе над сценой.

2. Создание проекта

2.1. Размещение простых объектов

Итак, начнем с создания объектов и размещения их в сцене. Первыми будут статические объекты — пол и стены. Затем выберем место для

источников света и камеры. Последним создадим игрока — это будет объект, к которому вы добавите сценарии, перемещающие его по сцене. Рисунок 1.12 демонстрирует вид редактора после завершения работы.

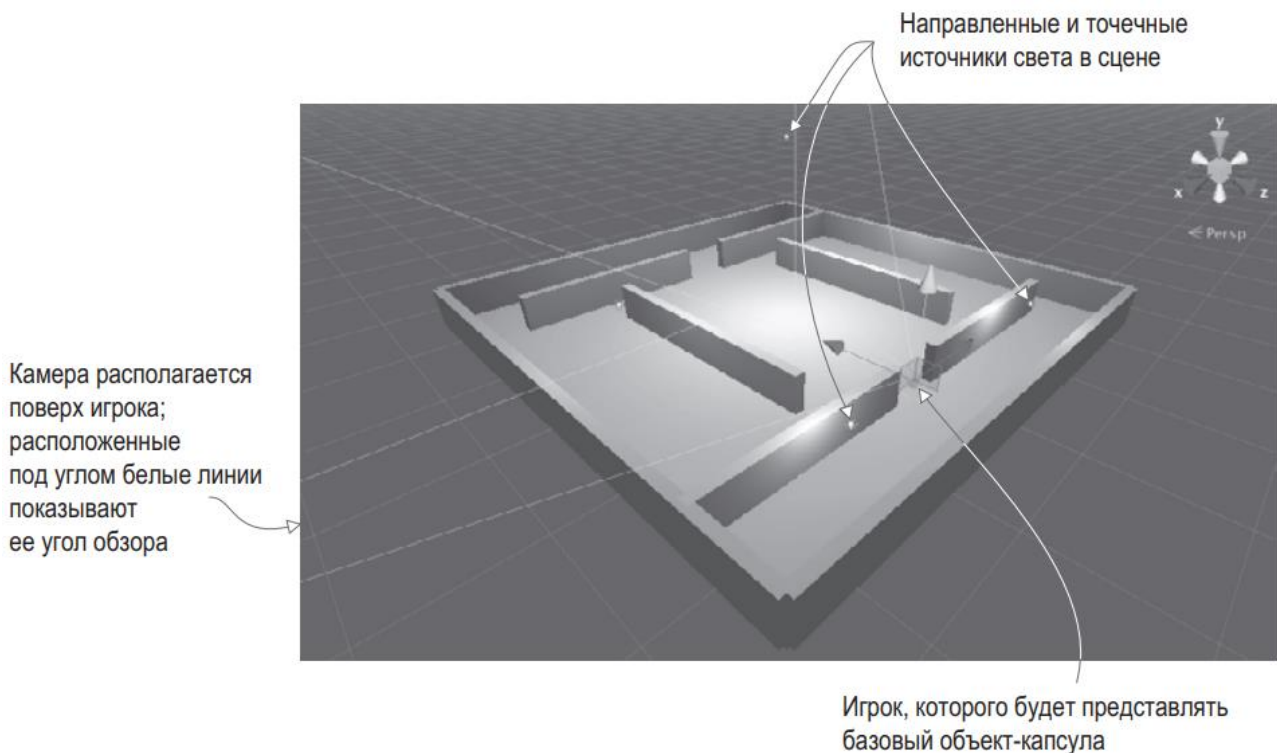


Рис. 2.1. Сцена в редакторе с полом, стенами, источниками света, камерой и игроком

Напоминаю, что нужно выбрать в меню **File** команду **New Project** и в появившемся окне указать имя проекта. После этого сразу же сохраните пустую сцену, так как изначально файл сцены у нового проекта отсутствует. Начнем мы с создания наиболее очевидных объектов.

2.2. Создание уровня. Декорации: пол, внешние и внутренние стены

В расположенном в верхней части экрана меню **GameObject** наведите указатель мыши на строчку **3D Object**, чтобы открыть дополнительное меню. Выберите в нем вариант **Cube**, так как для нашей сцены требуется куб (позднее вы поработаете и с другими фигурами, такими как **Sphere** и **Capsule**). Отредактируйте положение и масштаб появившегося в сцене куба, а также его имя таким образом, чтобы получить пол; значения, которые следует присвоить параметрам этого объекта на панели **Inspector**, показаны на рис. 2.2 (для превращения куба в пол его нужно растянуть).

1. Вверху в панели инспектора вводится имя объекта. Присвоим объекту, служащему полом, имя «**Floor**»

2. Передвиньте и отмасштабируйте куб, чтобы создать пол. В разной степени растянутый по разным осям объект теряет кубическую форму. При этом мы его слегка опускаем вниз для компенсации высоты, но объект остается центрированным относительно начала координат.

3. Это остальные компоненты объекта Cube, которые на данном этапе следует оставить без изменений. В эту группу попадают компоненты: Mesh Filter (определяющий геометрию объекта), Mesh Renderer (определяющий материал объекта) и Box Collider (позволяющий объекту принимать участие в столкновениях во время движения).

4. Повторите описанную последовательность для создания внешних стен комнаты. Можно каждый раз задействовать новый куб, а можно копировать и вставлять существующий объект, указывая стандартные сокращения (копировать объекты можно комбинацией клавиш **ctrl+D** – **Duplicate**). Двигайте, поворачивайте и масштабируйте стены, чтобы получить показанный на рис. 2.5 периметр. Экспериментируйте с различными значениями (например, 1, 4, 50 для полей Scale) или воспользуйтесь инструментами преобразований, с которыми вы познакомились в разделе 1.2.1 (напоминаю, что перемещения и повороты в трехмерном пространстве называют преобразованиями).

5. В результате у вас в панели иерархии должно быть несколько вариантов стен и элемент пола. Вы можете сгруппировать объекты, выделив их в панели иерархии и выбрав **Create Empty Parent**. Статические объекты, относящиеся к уровню или окружающей среде, рекомендуется объединять в группу **Environment**. Результат может выглядеть примерно так, как представлено на рис. 2.2.

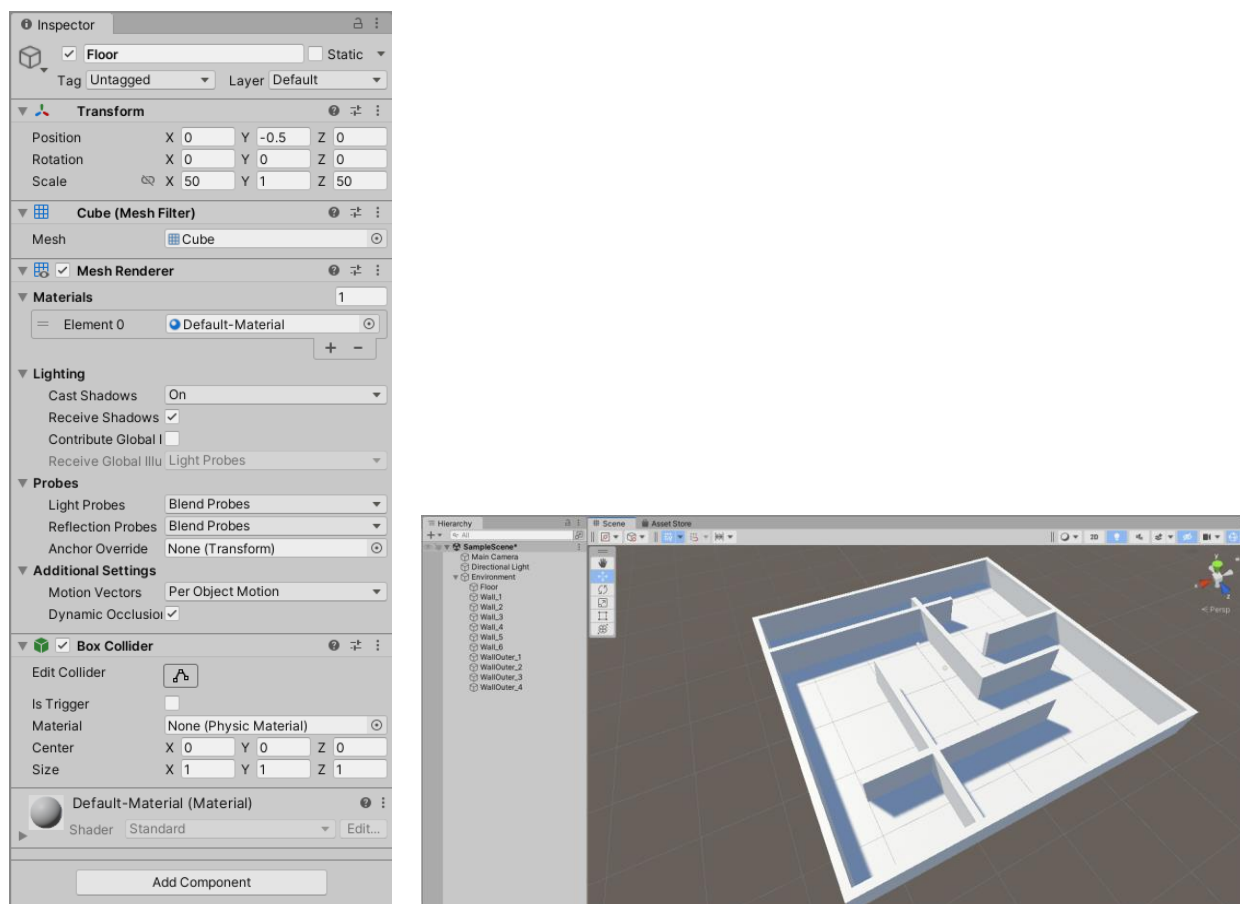


Рис. 2.2. Создание игрового окружения

Не забудьте про средства навигации, позволяющие рассматривать сцену под разными углами и менять ее масштаб, например имитируя взгляд с высоты птичьего полета. При этом нажатие клавиши F вернет вас к просмотру выделенного в данный момент объекта.

Точные значения преобразований для стен будут зависеть от того, каким образом вы повернете и отмасштабируете исходные объекты Cube, подогнав их размеры и положение, а также от способа их связывания на вкладке Hierarchy. Учитывайте, что если вы объединили дочерние объекты вокруг родительского Environment, то координаты стен и пола будут рассчитываться относительно исходных параметров Transform родительского объекта. Достоинством такого подхода является возможность редактировать параметры дочерних объектов, просто изменяя положение и масштаб родительского объекта (например, вы можете просто увеличить высоту стен изменив значение Scale Y у объекта Environment).

Что такое GAMEOBJECT?

Все объекты сцены представляют собой экземпляры класса GameObject аналогично тому, как все компоненты сценариев наследуют от класса MonoBehaviour. Этот факт становится более наглядным, если пустому объекту присвоить имя GameObject. Впрочем, даже если этот объект будет называться Floor, Camera или Player, суть не изменится.

На самом деле GameObject представляет собой всего лишь контейнер для набора компонентов. Его основным назначением является обеспечение некоего объекта, к которому можно присоединять класс MonoBehaviour. Как все это будет выглядеть в сцене, зависит от добавленных к объекту GameObject компонентов. К примеру, куб получается добавлением компонента Cube, сфера — добавлением компонента Sphere, и т. п.

2.2.2. Источники света и камеры

Как правило, трехмерные сцены освещаются направленным источником света, к которому добавляется набор точечных осветителей. Начать имеет смысл с направленного источника. Он может по умолчанию присутствовать в сцене, но если его нет, наведите указатель мыши на строку Light в меню GameObject и в открывшемся дополнительном меню выберите вариант Directional Light.

ТИПЫ ОСВЕТИТЕЛЕЙ

Существуют различные типы осветителей, разными способами проецирующие световые лучи. Три основных типа: **точечный источник**, **прожектор** и **направленный источник**.

Все лучи точечного источника (point light) начинаются в одной точке и распространяются во всех направлениях. В реальном мире таким осветителем является лампочка. Яркость света увеличивается по мере приближения к источнику за счет концентрации лучей.

Лучи прожектора (spot light) также исходят из одной точки, но распространяются в пределах ограниченного конуса. В текущем проекте мы с прожекторами работать не будем, но осветители данного типа повсеместно используются для подсвечивания отдельных частей уровня.

Лучи направленного источника света (directional light) распространяются равномерно и параллельно друг другу, одинаково освещая все элементы сцены. Это аналог солнца.

Испускаемый направленным осветителем свет не зависит от местоположения источника, значение имеет только его ориентация, поэтому его можно поместить в произвольную точку сцены. Рекомендуется установить его над комнатой, чтобы он выглядел как солнце и не мешал вам при работе с остальными фрагментами сцены. Поверните источник света и посмотрите, как это повлияет на освещенность комнаты; для получения нужного эффекта я рекомендую слегка повернуть его относительно осей X и Y. На панели Inspector вы найдете параметр Intensity (рис. 2.3). Как следует из его названия, он управляет яркостью света. Если бы данный направленный осветитель был в сцене единственным, его яркость имело бы смысл увеличить, но, так как мы добавим несколько точечных источников света, его можно оставить тусклым. Например, присвойте параметру Intensity значение 0.6. Вы также можете изменить цвет освещения и другие параметры.

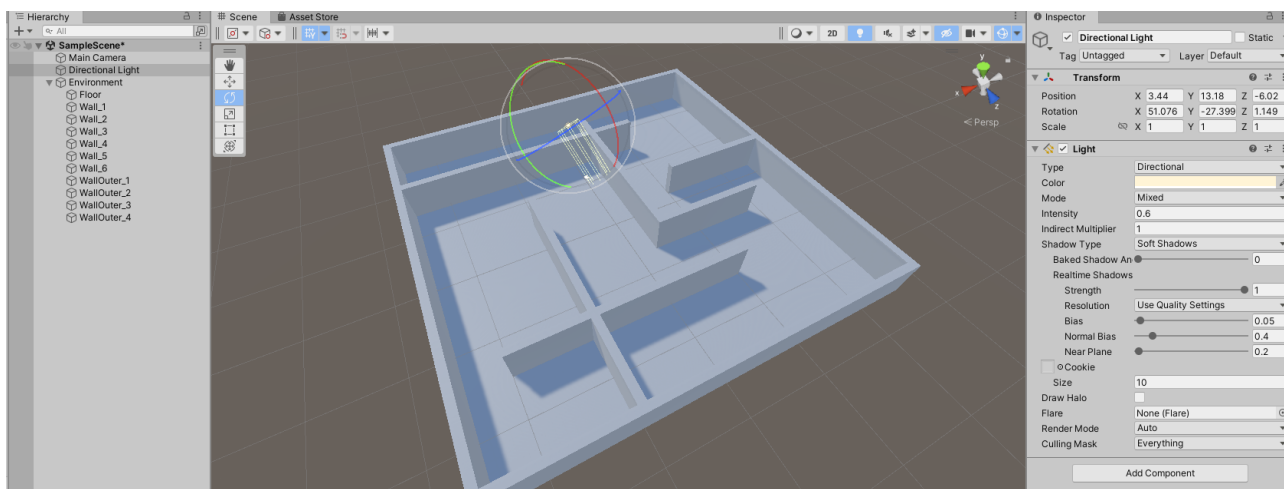


Рис. 2.3. Настройка направленного источника света на панели Inspector

Теперь командой **Create Light** создайте несколько точечных осветителей, поместив их в темные места комнаты. Вам нужно, чтобы все было как следует освещено. Источников должно быть не слишком много (иначе пострадает производительность), вполне достаточно одного осветителя в каждом углу (я предлагаю поднять их к верхней кромке стен) и одного, расположенного высоко над сценой (например, со значением 18 координаты Y). Это придаст освещению комнаты некое разнообразие. Обратите внимание, что у точечных источников света на панели Inspector появляется дополнительный параметр

Range (рис. 2.4). Он управляет дальностью распространения лучей. Если направленный источник света равномерно освещает всю сцену, то яркость точечного источника уменьшается по мере удаления от него. Поэтому для стоящего на полу источника этот параметр может быть равен 18, а для поднятого к верхней кромке стены его необходимо увеличить до 40, иначе он не сможет осветить всю комнату.

Обратите внимание на то, как реагируют источники света на изменение параметров: Intensity, Range, Color, Shadow Type, а также того располагается источник света выше стен или ниже.

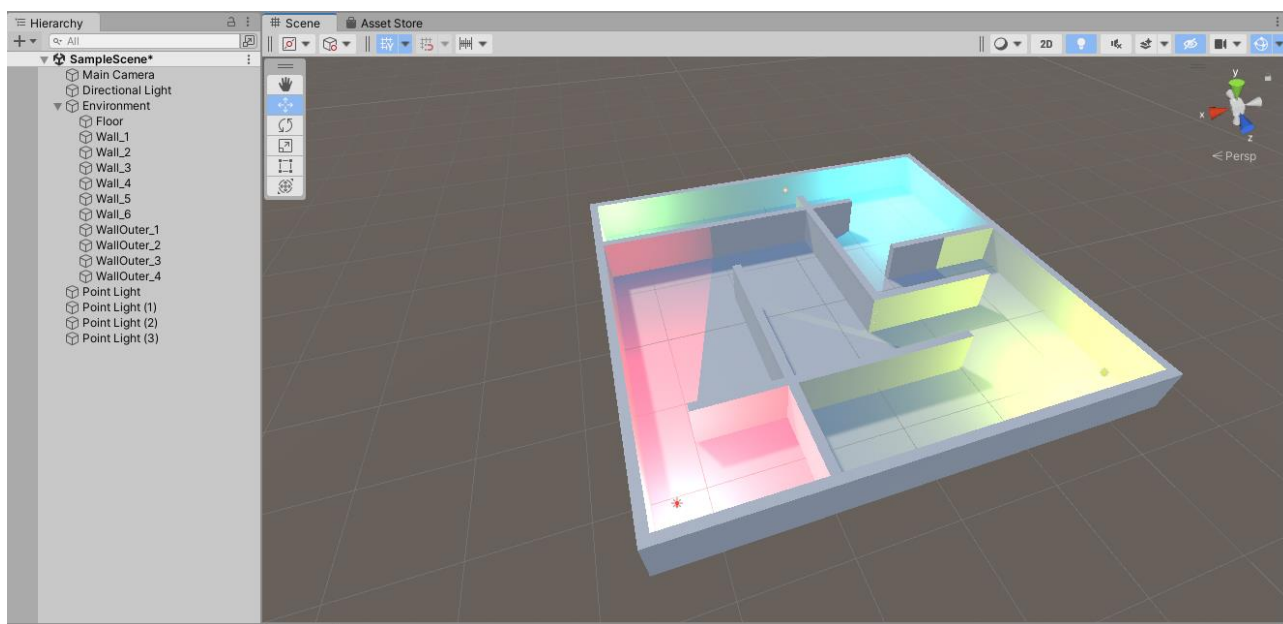


Рис. 2.4. Настройка точечных источников света

Чтобы игрок мог наблюдать за происходящим, требуется еще один тип объекта — камера. Впрочем, пустая сцена содержит основную камеру, которой мы можем воспользоваться. Если вам когда-нибудь потребуется создать дополнительные камеры (например, для разделения экрана в многопользовательских играх), выберите команду Camera в меню GameObject. Нашу же камеру мы расположим поверх игрока, чтобы наблюдать сцену его глазами.

2.2.3. Коллайдер и точка наблюдения игрока

В этом проекте игрока будет представлять обычный примитив. В меню GameObject (напоминаю, что для открытия этого дополнительного меню нужно навести указатель мыши на строку 3D Object) выберите вариант **Capsule**. Появится цилиндрическая фигура со скругленными концами — это и есть наш игрок. Сместите объект вверх, сделав его координату Y равной 1.1 (половина высоты объекта, плюс еще немного, чтобы избежать перекрывания с полом). Теперь наш игрок может произвольным образом перемещаться вдоль осей X и Z при условии, что он остается внутри комнаты и не касается стен. Присвойте

объекту имя **Player**. На панели Inspector вы увидите, что этому объекту назначен **капсульный коллайдер**. Это очевидный вариант для объекта Capsule, точно так же, как объект Cube по умолчанию обладает коллайдером Box. Но так как наша капсула будет представлять игрока, ее компоненты должны слегка отличаться от компонентов большинства объектов. Капсульный коллайдер мы удалим. Для этого нужно щелкнуть на значке с изображением шестерни справа от имени компонента, как показано на рис. 2.5. Откроется меню, в котором в числе прочих вы найдете и команду **Remove Component**. Коллайдер выглядит как окружающая объект зеленая сетка, поэтому после удаления компонента вы обнаружите, что она исчезла.

Коллайдер отвечает за то, как **физический движок** Unity будет взаимодействовать с **объектом игрока**, а именно где будут располагаться физические границы данного объекта.

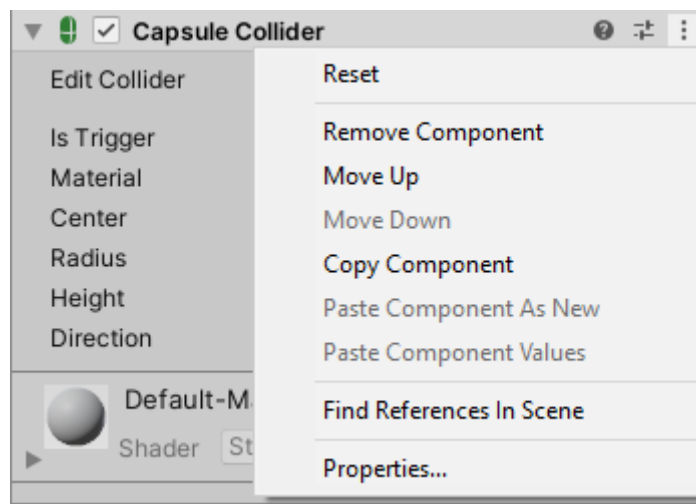


Рис. 2.5. Удаление компонента на панели Inspector

Вместо капсульного коллайдера мы назначим объекту контроллер персонажа. В нижней части панели Inspector вы найдете кнопку **Add Component**. Щелчок на ней открывает меню с перечнем типов доступных компонентов. В разделе **Physics** и находится нужная нам строка **Character Controller**. Как несложно догадаться, именно этот компонент позволит объекту вести себя как персонаж.

Для завершения настройки игрока осталось сделать всего один шаг — присоединить к нему камеру. Как уже упоминалось в разделе, посвященном созданию пола и стен, вы можете перетаскивать объекты и класть их друг на друга на вкладке Hierarchy. Прodelайте эту операцию, **перетащив камеру MainCamera на капсулу**, чтобы присоединить ее к игроку. Затем расположите ее таким образом, чтобы она соответствовала глазам игрока (я предлагаю указать координаты 0, 0.5, 0). При необходимости верните координатам преобразования поворота значения 0, 0, 0 (если вы вращали капсулу, они будут различаться). При желании, вы можете расположить камеру за объектом игрока

(сзади-сверху), и наклонить угол обзора чуть вниз, тогда игра примет вид от третьего лица. Впрочем, в дальнейшем можно столкнуться с другими проблемами, например при соприкосновении объекта со стеной.

Итак, в сцене присутствуют все необходимые объекты. Осталось написать код перемещения игрока.

2.3. Двигаем объекты: сценарий, активирующий преобразования

Чтобы заставить игрока перемещаться по сцене, нам потребуются сценарии движения, которые будут присоединены к игроку. Напоминаю, что компонентами называются модульные фрагменты функциональности, добавляемые к объектам, поэтому сценарии тоже можно считать своего рода компонентами. Именно они будут реагировать на клавиатурный ввод и манипуляции мышью, но для начала мы заставим игрока поворачиваться на месте. Это продемонстрирует вам, как применять преобразования в коде. Напомню, что преобразования бывают трех видов — перемещение, поворот и масштабирование; вращение объекта на месте соответствует преобразованию поворота. Но вы должны знать об этой задаче еще кое-что, кроме того, что она «сводится к изменению ориентации объекта».

2.3.1. Схема программирования движения

Анимация объекта (например, его вращение) сводится к смещению его в каждом кадре на небольшое расстояние и к последующему многократному воспроизведению всех кадров. Само по себе преобразование происходит мгновенно, в отличие от движения, растянутого во времени. Но последовательное применение набора преобразований вызывает визуальное перемещение объекта, совсем как набор рисунков в кинеографе. Этот принцип проиллюстрирован на рис. 2.6.

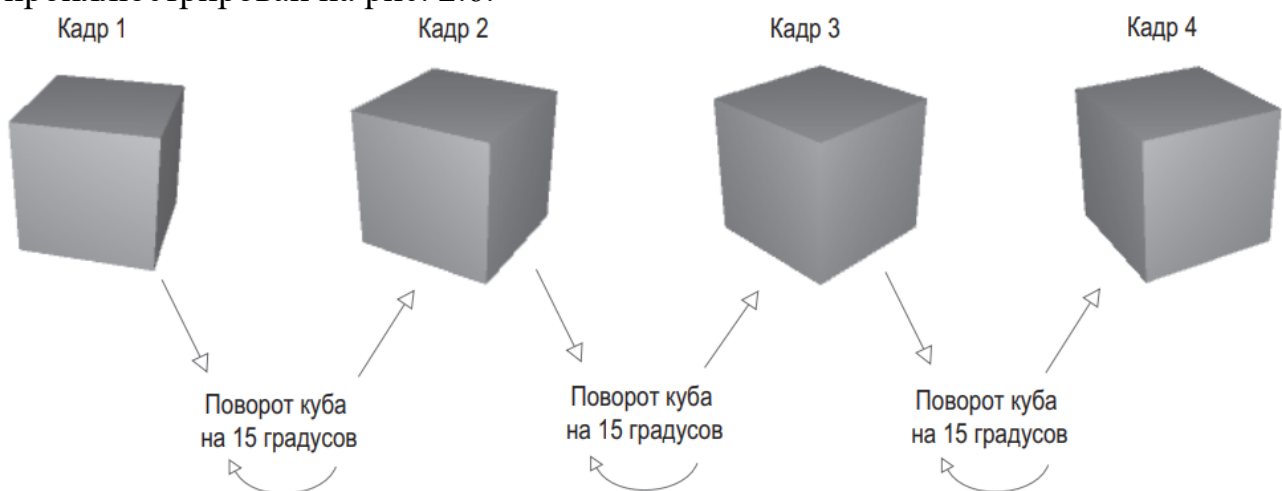


Рис. 2.6. Циклическое применение последовательности преобразований к объекту для возникновения движения

Компоненты сценария снабжены выполняющимся в каждом кадре методом **Update()**. Поэтому, чтобы заставить куб вращаться вокруг своей оси,

следует добавить в метод Update() код, поворачивающий его на небольшой угол. Этот код будет запускаться в каждом кадре снова и снова.

Применим рассмотренную концепцию на практике. Создайте новый сценарий командой C# script (напоминаю, что она находится в дополнительном меню Create, доступ к которому осуществляется через меню Assets), присвойте ему имя Spin и напишите код из следующего листинга (не забудьте после ввода листинга сохранить файл).

Листинг 2.1. Приводим объект во вращение

```
public class Spin : MonoBehaviour
{
    public float speed = 3.0f; // Объявление общедоступной переменной для скорости вращения.
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        transform.Rotate(0, speed, 0); // Поместите сюда команду Rotate, чтобы она запускалась в каждом кадре.
    }
}
```

Для добавления компонента сценария к игроку перетащите сценарий с вкладки Project на строку Player на вкладке Hierarchy. Щелкните на кнопке Play, и вы увидите, как все придет во вращение — вы написали код, заставляющий объект двигаться. В основном этот код состоит из шаблона нового сценария, к которому добавлены две строки. Посмотрим, что именно они делают.

Прежде всего, появилась переменная для скорости, добавленная в верхнюю часть определения класса. Превращение скорости вращения в переменную произошло по двум причинам. Первая — это соблюдение стандартного правила программирования: «никаких магических чисел». Вторая же причина связана со способом отображения общедоступных переменных в Unity. Общедоступные переменные появляются на панели Inspector, что позволяет редактировать значения уже добавленных к игровому объекту компонентов. Это называется «сериализацией» значения, так как Unity сохраняет модифицированное состояние переменной.

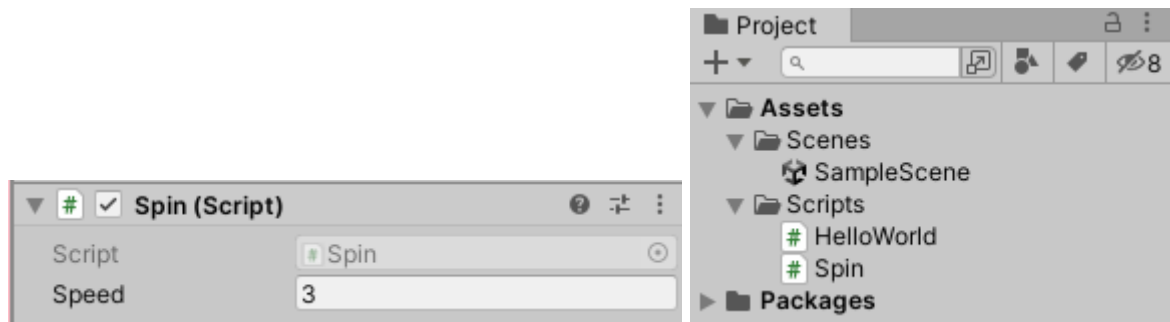


Рис. 2.7. Добавление сценария Spin к объекту Player вызывает появление новой сериализованной переменной Speed, значение которой можно изменять непосредственно из интерфейса Inspector (если при этом игра запущена, то изменения не сохраняются, что удобно при тестировании различных параметров). Сценарий Spin создается в панели проекта нажатием кнопки Create C# Script

Вторая строка, на которую следует обратить внимание в листинге 2.1, касается метода **Rotate()**. Он вставлен в метод `Update()`, а значит, выполняется в каждом кадре. Так как метод `Rotate()` принадлежит классу `Transform`, он вызывается при помощи точечной нотации через компоненты преобразования объекта (как и в большинстве объектно-ориентированных языков, если вы введете **transform**, это будет восприниматься как **this.transform**). В рассматриваемом случае преобразование сводится к повороту на **speed** градусов в каждом кадре, что в итоге даст нам плавное вращение.

В трехмерном пространстве есть три оси: X, Y и Z. Интуитивно понятно, каким образом они связаны с местоположением объекта и его перемещениями, но, кроме того, их применяют для описания преобразования поворота. Сходным образом описывают вращение в воздухоплавании, поэтому работающие с трехмерной графикой программисты зачастую пользуются терминами из механики полета: тангаж (pitch), рысканье (yaw) и крен (roll); тангаж означает вращение вокруг оси X, рысканье — вращение вокруг оси Y, крен — вращение вокруг оси Z.

Возможность описывать вращение вокруг осей X, Y и Z означает три параметра для метода `Rotate()`. Так как нам требуется только вращение вокруг своей оси без наклонов вперед и назад, меняться должна только координата Y, координатам же X и Z мы просто присвоим значение 0. Существует еще одна тонкость, связанная с вращением и осями координат, реализованная в виде необязательного четвертого параметра метода `Rotate()`.

2.3.3. Локальные и глобальные координаты

По умолчанию метод `Rotate()` работает с так называемыми **локальными координатами**, хотя вы также можете использовать глобальную систему координат. Вы сообщаете методу, какой именно системой координат хотите воспользоваться, при помощи четвертого необязательного параметра, который может принимать два значения: `Space.Self` и `Space.World`. Например:

```
Rotate(0, speed, 0, Space.World)
```

Вернитесь к описанию трехмерного координатного пространства и подумайте над следующими вопросами: где находится точка (0, 0, 0)? как выбирается положительное направление оси X? может ли перемещаться сама координатная система?

Оказывается, у каждого объекта есть собственная точка начала координат и собственные положительные направления осей. И эта координатная система перемещается вместе с объектом. В таких случаях говорят о локальных координатах. При этом трехмерная сцена как целое обладает собственной точкой начала координат и собственными направлениями осей, причем такая координатная система всегда статична. Это так называемые глобальные координаты. Соответственно, указывая вариант системы координат для метода Rotate(), вы сообщаете, относительно каких осей X, Y и Z нужно выполнить преобразование поворота.

Новичкам в области трехмерной графики эта концепция может показаться сложной для понимания. Различные оси показаны на рис. 2.8 (обратите внимание, что, к примеру, понятие «лево» для самолета отличается от понятия «лево» для сцены в целом), но разницу между локальными и глобальными координатами проще всего понять на примере.

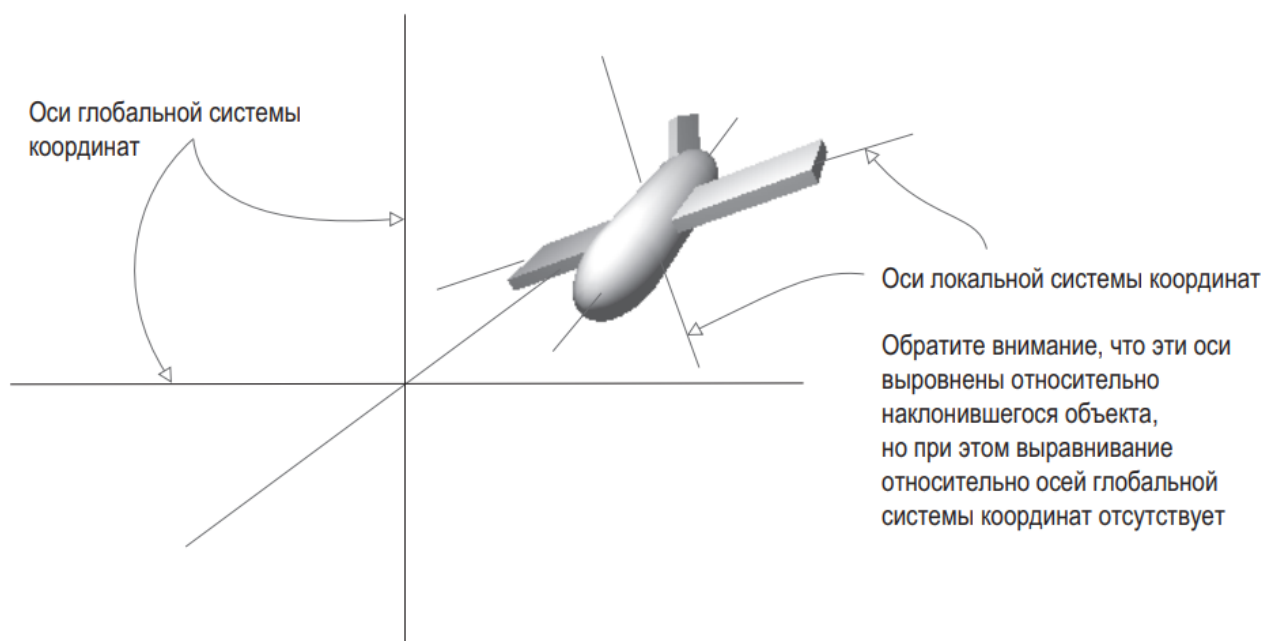


Рис. 2.8. Оси локальной и глобальной систем координат

2.4. Компонент сценария для осмотра сцены: MouseLook

Теперь нужно заставить преобразование поворота реагировать на ввод с помощью мыши. В данном случае подразумевается вращение объекта, к которому присоединен сценарий, то есть игрока. Задача будет решаться в несколько этапов путем постепенного добавления персонажу двигательных возможностей. Сначала мы заставим игрока поворачиваться только из стороны

в сторону, а затем — только вверх и вниз. В конечном счете игрок научится смотреть во всех направлениях (поворачиваясь одновременно в горизонтальной и вертикальной плоскостях). Такое поведение называют **слежением за мышью (mouse-look)**.

Так как у нас предполагается три типа поведения при повороте (по горизонтали, по вертикали и комбинированный), начнем мы с написания фреймворка, поддерживающего все эти типы. Создайте новый сценарий на C# с именем `MouseLook` и добавьте в него код из следующего листинга.

Листинг 2.2. Сценарий `MouseLook` с перечислением для преобразования поворота

```
public class MouseLook : MonoBehaviour
{
    public enum RotationAxes { // Объявляем структуру данных enum, которая будет
        // сопоставлять имена с параметрами.
        MouseXAndY = 0,
        MouseX = 1,
        MouseY = 2
    }

    public RotationAxes axes = RotationAxes.MouseXAndY; // Объявляем общедоступную
    // переменную, которая появится в редакторе Unity.

    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
        if (axes == RotationAxes.MouseX) {
            // это поворот в горизонтальной плоскости
        }
        else if (axes == RotationAxes.MouseY) {
            // это поворот в вертикальной плоскости
        }
        else {
            // это комбинированный поворот → Сюда поместим код для комбинированного
            // вращения.
        }
    }
}
```

Обратите внимание, что именно перечисление позволяет сценарию `MouseLook` выбирать, каким образом — в горизонтальной или вертикальной плоскости — будет выполняться поворот. Определив структуру данных *enum*, вы получите возможность задавать значения по имени вместо того, чтобы

указывать числа и пытаться запомнить, что означает каждое из них. (Означает ли ноль вращение по горизонтали? Или такому повороту соответствует единица?) Если затем объявить общедоступную переменную, типизированную как перечисление, она появится на панели Inspector в виде раскрывающегося меню, удобного для выбора параметров.

Удалите компонент Spin (тем же способом, которым ранее удалялся капсульный коллайдер) и вместо него присоедините к игроку новый сценарий.

2.4.1. Горизонтальное вращение, следящее за указателем мыши

Первая и наиболее простая ветка соответствует вращению в горизонтальной плоскости. Для начала напишем уже знакомую вам команду из листинга 2.1, заставляющую объект поворачиваться. Не забудьте объявить общедоступную переменную для скорости вращения; сделайте это после объявления осей, но до метода Update(), назвав новую переменную sensitivityHor, так как слово «speed» имеет слишком общий смысл и не позволит отличать варианты поворота друг от друга. Увеличьте значение этой переменной до 9, так как совсем скоро она начнет масштабироваться. Исправленный код приведен в следующем листинге.

Листинг 2.3. Поворот по горизонтали, пока не связанный с движениями указателя

```
void Update()
{
    if (axes == RotationAxes.MouseX) { // это поворот в горизонтальной
плоскости
        if (axes == RotationAxes.MouseX) {
            transform.Rotate(0, sensitivityHor, 0); // Сюда мы поместим команду
Rotate, чтобы она запускалась в каждом кадре
        }
    }
}
```

Если запустить сценарий, сцена, как и раньше, начнет вращаться, но намного быстрее, потому что теперь скорость вращения вокруг оси Y равна 9, а не 3. Теперь нам нужно сделать так, чтобы преобразование возникало в ответ на движения указателя мыши, поэтому создадим новый метод: **Input.GetAxis()**. Класс Input обладает множеством методов для обработки информации, поступающей с устройств ввода (таких, как мышь). В частности, метод **GetAxis()** возвращает числа, связанные с движениями указателя мыши (положительные или отрицательные в зависимости от направления движения). Он принимает в качестве параметра имя нужной оси. А горизонтальная ось у нас называется Mouse X.

Если умножить скорость вращения на координату оси, объект начнет поворачиваться вслед за указателем мыши. Скорость масштабируется в соответствии с перемещениями указателя, уменьшаясь до нуля и даже меняет направление. Новый вид команды Rotate показан в следующем листинге.

Листинг 2.4. Команда Rotate, реагирующая на движения указателя мыши

```
...
transform.Rotate(0, Input.GetAxis("Mouse X") * sensitivityHor, 0);
// Метод GetAxis() получает данные, вводимые с помощью мыши.
...
```

Обратите внимание, что для того, чтобы увидеть результат работы в данный момент, потребуется изменить значение Axes в поле инспектора объекта Player на Mouse X и выбрать активным окно Play после запуска игры. Можете также поэкспериментировать со значением Sensitivity Hor. Щелкните на кнопке Play и подвигайте мышью в разные стороны. Объект начнет поворачиваться вправо и влево вслед за указателем. Теперь нужно научить объект игрока вращаться в вертикальной плоскости.

2.4.2. Упрощенный поворот по вертикали

Вращение в горизонтальной плоскости было реализовано с помощью метода Rotate(), но для поворота по вертикали мы воспользуемся другим способом. Дело в том, что указанный метод при всем его удобстве в осуществлении преобразований недостаточно гибок. Он применим только для неограниченного приращения угла поворота, что в нашей ситуации подходит только для вращения в горизонтальной плоскости. В случае же поворота по вертикали требуется задать предел наклона вниз и вверх. Код этого преобразования для сценария MouseLook представлен в следующем листинге.

Листинг 2.5. Поворот в вертикальной плоскости для сценария MouseLook

```
...
public RotationAxes axes = RotationAxes.MouseXAndY;
    public float sensitivityVert = 9.0f; // Объявляем переменные, задающие поворот в вертикальной
плоскости
    public float minimumVert = -45.0f;
    public float maximumVert = 45.0f;
    private float _rotationX = 0; // Объявляем закрытую переменную для угла поворота по
вертикали
    public float sensitivityHor = 9.0f;

    //...

    // Update is called once per frame
    void Update()
    {
        if (axes == RotationAxes.MouseX) {
            if (axes == RotationAxes.MouseX) {
                transform.Rotate(0, Input.GetAxis("Mouse X") * sensitivityHor, 0);
            }
        }
        else if (axes == RotationAxes.MouseY) {
            _rotationX -= Input.GetAxis("Mouse Y") * sensitivityVert; // Увеличиваем угол
поворота по вертикали в соответствии с перемещениями указателя мыши.
        }
    }
}
```

```

        _rotationX = Mathf.Clamp(_rotationX, minimumVert, maximumVert); // Фиксируем угол
поворота по вертикали в диапазоне, заданном минимальным и максимальным значениями.
        float rotationY = transform.localEulerAngles.y; // Сохраняем одинаковый угол поворота
вокруг оси Y (т. е. вращение в горизонтальной плоскости отсутствует).
        transform.localEulerAngles = new Vector3(_rotationX, rotationY, 0); // Создаем новый
вектор из сохраненных значений поворота.
    }
}

```

Выбираем в меню Axes компонента MouseLook вращение по вертикали и воспроизводим новый сценарий. Теперь сцена вместо поворотов из стороны в сторону будет вслед за движениями указателя мыши наклоняться вверх и вниз. При этом углы поворота ограничиваются указанными вами пределами.

Этот код знакомит вас с рядом новых понятий, которые следует объяснить более подробно. Во-первых, теперь мы не пользуемся методом Rotate(), поэтому нам требуется еще одна переменная (она называется **_rotationX**, так как вертикальное вращение происходит вокруг оси X), предназначенная для сохранения угла поворота. Метод Rotate() просто увеличивает угол поворота, в то время как на этот раз мы его задаем в явном виде. Другими словами, если в предыдущем листинге мы просили «добавить 5 к текущему значению угла», то теперь мы «присваиваем углу поворота значение 30». Разумеется, нам и в этом случае нужно увеличивать угол поворота, именно поэтому в коде присутствует оператор `+=`. То есть мы вычитаем значение из угла поворота, а не присваиваем это значение углу. Так как методом Rotate() мы не пользуемся, манипулировать углом поворота можно разными способами, а не только увеличивая его. Этот параметр умножается на Input.GetAxis(), совсем как в коде для вращения в горизонтальной плоскости, просто на этот раз мы выясняем значение Mouse Y, ведь нас интересует вертикальная ось.

Следующая строка также посвящена манипуляциям углом поворота. Метод **Mathf.Clamp()** позволяет ограничить этот параметр максимальным и минимальным значениями. Эти предельные значения задаются объявленными общедоступными переменными, которые гарантируют поворот сцены вверх и вниз всего на 45 градусов. Метод **Clamp()** работает не только с преобразованиями поворота. Он в принципе используется для сохранения числового значения переменной в заданных пределах. В экспериментальных целях превратите строку с методом Clamp() в комментарий; вы увидите, как объект начнет свободно вращаться в вертикальной плоскости. Понятно, что нам совсем не обязательно смотреть на сцену, перевернутую с ног на голову; отсюда и появляются ограничения.

Так как угловое свойство преобразования выражается переменной **Vector3**, мы должны создать новую переменную с углом поворота, которая будет передаваться в конструктор. Метод Rotate() этот процесс автоматизировал, увеличив угол поворота и затем создав новый вектор.

Вектором (*vector*) называется набор чисел, сохраняемых как единое целое. К примеру, **Vector3** состоит из трех чисел (помеченных как *x*, *y*, *z*).

Мы создаем новый вектор **Vector3**, вместо того чтобы поменять значения у существующего, так как в случае преобразований эти значения предназначены только для чтения. Это крайне распространенная ошибка.

Углы Эйлера и Кватернионы

Свойство объекта, к которому мы обращаемся, называется **localEulerAngles**. Кватернионы представляют собой математическую конструкцию для представления вращения объектов. Они отличаются от углов Эйлера, то есть используемого нами подхода с осями *X*, *Y*, *Z*. Помните обсуждение тангажа, рысканья и крена? В этом случае вращения были представлены с помощью углов Эйлера. Если вам требуется детальное объяснение, попробуйте прочесть документ <http://wat.gamedev.ru/articles/quaternions>. Несколько проще объяснить, почему кватернионы используются для представления вращений: они более равномерно реализуют интерполяцию между углами поворота (то есть переход от одного промежуточного значения к другому). Свойство **localRotation** выражено через кватернионы, а не через углы Эйлера. При этом в Unity существует и свойство, выражаемое через углы Эйлера, благодаря которому понять, как именно происходит процесс поворота, становится проще; преобразование одного варианта значений в другой и обратно делается автоматически. Инструмент Unity выполняет все сложные вычисления в фоновом режиме, избавляя вас от необходимости проделывать их вручную.

Осталось написать код для последнего вида вращения, происходящего одновременно в горизонтальной и вертикальной плоскостях.

2.4.3. Одновременные горизонтальное и вертикальное вращения

В последнем фрагменте кода метод **Rotate()** также не применяется, так как нам снова требуется ограничить диапазон углов поворота по вертикали. Это означает, что горизонтальный поворот тоже нужно вычислять вручную. Напоминаю, что метод **Rotate()** автоматизировал процесс приращения угла поворота.

Листинг 2.6. Сценарий **MouseLook**, поворачивающий объект одновременно по горизонтали и по вертикали

```
else { // это комбинированный поворот - Сюда поместим код для комбинированного вращения.
    _rotationX -= Input.GetAxis("Mouse Y") * sensitivityVert;
    _rotationX = Mathf.Clamp(_rotationX, minimumVert, maximumVert);
    float delta = Input.GetAxis("Mouse X") * sensitivityHor; // Приращение угла поворота
    // через значение delta.
    float rotationY = transform.localEulerAngles.y + delta; // Значение delta - это
    // величина изменения угла поворота.

    transform.localEulerAngles = new Vector3(_rotationX, rotationY, 0);
}
```

Первая пара строк, посвященная переменной `_rotationX`, полностью аналогична таким же строкам из предыдущего сценария. Просто помните, что вращение в вертикальной плоскости происходит вокруг оси `X` объекта. А так как вращение в горизонтальной плоскости больше не реализуется через метод `Rotate()`, появились строки с переменными **delta** и **rotationY**. Дельта — это распространенное математическое обозначение «величины изменения», поэтому наше вычисление переменной **delta** дает величину, на которую следует поменять угол поворота. Затем она добавляется к текущему значению этого угла для придания объекту желаемой ориентации.

В конце оба угла наклона — по вертикали и по горизонтали — используются для создания нового вектора, который назначается угловому свойству компонента **Transform**.

2.4.4. Отключаем влияние среды на игрока. Компонент **Rigidbody**

Хотя для данного проекта это не имеет особого значения, в большинстве современных игр от первого лица на все элементы сцены действует модель физической среды. Она заставляет объекты отскакивать и опрокидываться; такое поведение подходит для большинства объектов, но вращение нашего игрока должно контролироваться исключительно мышью, не попадая под влияние смоделированной физической среды. По этой причине в сценариях, где присутствует ввод с помощью мыши, к компоненту **Rigidbody** игрока обычно добавляют свойство **freezeRotation**. Поместите в сценарий **MouseLook** вот такой метод `Start()`:

```
void Start()
{
    Rigidbody body = GetComponent<Rigidbody>();
    if (body != null) // Проверяем, существует ли этот компонент.
        body.freezeRotation = true;
}
```

Rigidbody — это дополнительный компонент, которым может обладать объект. Моделируемая физическая среда влияет на этот компонент и управляет объектами, к которым он присоединен.

2.5. Компонент для клавиатурного ввода

Возможность смотреть по сторонам в ответ на перемещения указателя мыши относится к важным фрагментам элементов управления персонажем в играх от первого лица, но это только половина дела. Кроме этого, игрок должен перемещаться по сцене в ответ на клавиатурный ввод. Поэтому в дополнение к компоненту для элемента управления мышью напомним компонент для клавиатурного ввода; создайте новый компонент **C# script** с именем **FPSInput** и присоедините его к игроку (в дополнение к сценарию **MouseLook**). При этом мы на время ограничим компонент **MouseLook** только горизонтальным вращением.

Рассматриваемые нами элементы управления клавиатурным вводом и вводом с помощью мыши помещены в отдельные сценарии. Вы вовсе не обязаны структурировать код подобным образом и вполне можете поместить все в единый сценарий «элементов управления игроком». Но, как правило, модульные системы (такие, как в Unity) стремятся к максимальной гибкости, и, соответственно, эффективнее всего моделировать функциональность как набор отдельных небольших компонентов.

При разработке в Unity как правило используется **компонентный подход**, когда для конкретного объекта (например игрока) создается свой класс и сценарий, описывающий всё его поведение. Тем не менее, наличие отдельных модулей, отвечающих за управление объектами или работу с камерой, позволит подключать их также к другим объектам. Выбор используемого подхода остается за разработчиком и гейм-дизайнером.

Код, который мы писали, влиял только на ориентацию объекта, теперь же мы будем менять только его местоположение. Как показано в листинге 2.7, мы берем код вращения и вводим его в сценарий FPSInput, заменяя метод Rotate() методом Translate(). После щелчка на кнопке Play сцена начнет скользить вверх, а не вращаться, как раньше. Попробуйте поменять значения параметров и посмотреть, как изменится движение (например, после того как вы меняете местами первое и второе числа); после некоторого количества экспериментов можно будет перейти к добавлению клавиатурного ввода.

Листинг 2.7. Применение кода вращения для управления движением

```
public class FPSInput : MonoBehaviour
{
    public float speed = 6.0f; // Необязательный элемент на случай, если вы
    захотите изменить скорость.
    // ...
    // Update is called once per frame
    void Update()
    {
        transform.Translate(0, speed, 0); // Меняем метод Rotate() на метод
        Translate()
    }
}
```

2.5.1. Реакция на нажатие клавиш

Код перемещения в ответ на нажатия клавиш (показанный в листинге 2.8) аналогичен коду вращения в ответ на движение указателя мыши. Мы снова используем метод GetAxis(), причем аналогичным образом:

Листинг 2.8. Изменение положения игрока в ответ на нажатие клавиш

```
void Update()
{
    // transform.Translate(speed, 0, 0); // Меняем метод Rotate() на метод Translate()
```

```
float deltaX = Input.GetAxis("Horizontal") * speed;
float deltaZ = Input.GetAxis("Vertical") * speed;

transform.Translate(deltaX, 0, deltaZ);
}
```

Как и раньше, значения, задаваемые методом `GetAxis()`, умножаются на скорость, давая в итоге величину смещения. Но если раньше в ответ на запрос оси мы получали ответ «Mouse имя оси», то теперь это значения **Horizontal** или **Vertical**. Это абстрактные представления для параметров ввода в Unity; если в меню **Edit** навести указатель мыши на строку **Project Settings** и выбрать в открывшемся меню команду **Input**, откроется список абстрактных имен ввода и соответствующих им элементов управления (рис. 2.9). Клавиши с указывающими влево/вправо стрелками и с буквами A/D соответствуют имени `Horizontal`, в то время как клавиши со стрелками, указывающими вверх/вниз, и с буквами W/S соответствуют имени `Vertical`.

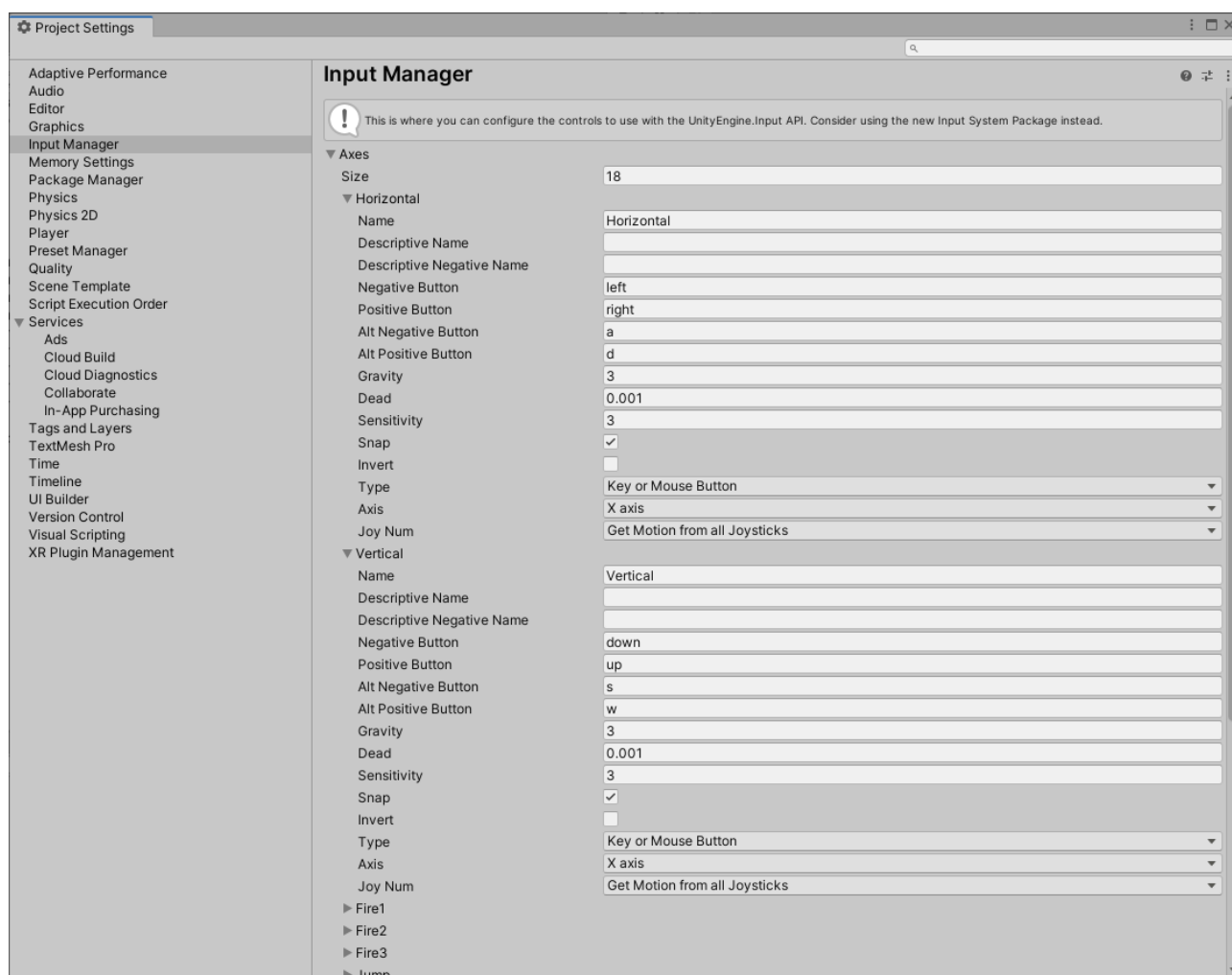


Рис. 2.9. Панель Input Manager для определения настроек устройств ввода в проекте

Обратите внимание, что значения перемещения меняются по координатам X и Z. Вы могли заметить в процессе экспериментов с методом `Translate()`, что

изменение координаты *X* соответствует движению из стороны в сторону, в то время как изменение координаты *Z* означает движение вперед и назад.

Вставив в сценарий этот новый код перемещения, вы получите возможность двигать объект по сцене нажатием клавиш со стрелками или клавиш с буквами WASD. Это стандартная ситуация для большинства игр от первого лица. Сценарий, управляющий перемещениями, практически готов, осталось только внести в него некоторые изменения.

Если вы запустите проект сейчас, то увидите, что игрок может свободно перемещаться через стены и пол, то есть, по сути, представляет собой **летающую камеру**, а не объект персонажа. Формально это происходит так как, у нас отключено взаимодействие объекта **Player** с физической системой Unity и объектами в сцене.

2.5.2. Независимая от скорости работы компьютера скорость перемещений

Пока вы запускаете код только на собственном компьютере, этот аспект незаметен, но на разных машинах игрок будет перемещаться с разной скоростью. Ведь более мощные компьютеры быстрее обрабатывают код и графику. В настоящее время скорость перемещения вашего игрока привязана к скорости работы компьютера. Это называется зависимостью от кадровой частоты (*frame rate dependent*), так как код движения зависит от частоты кадров игры.

Например, представьте, что вы запускаете деморолик на двух компьютерах, один из которых отображает на мониторе 30 кадров в секунду, а второй 60. Это означает, что на втором компьютере метод **Update()** будет вызываться в два раза чаще, двигая объект с одной и той же скоростью 6 при каждом вызове. В итоге при частоте 30 кадров в секунду скорость движения составит 180 единиц в секунду, в то время как частота 60 кадров в секунду обеспечит скорость перемещения в 360 единиц в секунду. Такая вариативность неприемлема для большинства игр.

Решить эту проблему позволяет такое редактирование кода, в результате которого мы получим независимость от кадровой частоты (*frame rate independent*). То есть скорость перемещения перестанет зависеть от частоты кадров игры. Это обеспечивается сменой скорости в соответствии с частотой кадров. Она должна уменьшаться или увеличиваться в зависимости от того, насколько быстро работает компьютер. Достигнуть такого результата нам поможет умножение скорости на переменную **deltaTime**, как показано в следующем листинге.

Листинг 2.9. Движение с независимостью от кадровой частоты благодаря переменной **deltaTime**

```
void Update()
{
    // transform.Translate(speed, 0, 0); // Меняем метод Rotate() на метод Translate()
    float deltaX = Input.GetAxis("Horizontal") * speed;
```

```

float deltaZ = Input.GetAxis("Vertical") * speed;

transform.Translate(deltaX * Time.deltaTime, 0, deltaZ * Time.deltaTime); // Добавляем
обращение к методу DeltaTime класса Time
//transform.Translate(deltaX, 0, deltaZ);
}

```

Класс **Time** обладает рядом свойств и методов, позволяющих регулировать время. К таким свойствам относится и **deltaTime**. Так как мы знаем, что дельта указывает на величину изменения, переменная **deltaTime** означает величину изменения во времени. А конкретно — это время между кадрами. Его величина зависит от частоты кадров (например, при частоте 30 кадров в секунду **deltaTime** составляет 1/30 секунды). Поэтому умножение скорости на эту переменную приведет к масштабированию скорости на различных компьютерах. Теперь движение нашего персонажа одинаково на всех машинах. Но сценарий еще не завершен; ведь, перемещаясь по комнате, мы можем проходить сквозь стены.

2.5.3. Компонент **CharacterController** для распознавания столкновений

Назначенные объекту напрямую преобразования не затрагивают такого аспекта, как распознавание столкновений. В результате персонаж начинает ходить сквозь стены. Поэтому нам требуется компонент **CharacterController**. Именно он обеспечит естественность перемещений нашего персонажа. Напомню, что в момент настройки игрока мы присоединяли этот компонент, поэтому теперь остается воспользоваться им в коде сценария **FPSInput** (листинг 2.10). Альтернативным подходом для реализации коллизий и взаимодействия одних объектов с другими является добавление компонента **Rigidbody** (твердое тело) к объекту, тогда на него начинает действовать физическая система Unity.

Листинг 2.10. Перемещение компонента **CharacterController** вместо компонента **Transform**

```

public class FPSInput : MonoBehaviour
{
    public float speed = 6.0f; /
    private CharacterController _charController; // Переменная для ссылки на компонент
    CharacterController.
    void Start()
    {
        _charController = GetComponent<CharacterController>(); // Доступ к другим компонентам,
        присоединенным к этому же объекту.
    }

    void Update()
    {
        float deltaX = Input.GetAxis("Horizontal") * speed;
        float deltaZ = Input.GetAxis("Vertical") * speed;

        Vector3 movement = new Vector3(deltaX, 0, deltaZ);
        movement = Vector3.ClampMagnitude(movement, speed); // Ограничим движение по диагонали
        той же скоростью, что и движение параллельно осям.

        movement *= Time.deltaTime;
    }
}

```

```

        movement = transform.TransformDirection(movement); // Преобразуем вектор движения от
        локальных к глобальным координатам.
        _charController.Move(movement); // Заставим этот вектор перемещать компонент
        CharacterController.

        //transform.Translate(deltaX * Time.deltaTime, 0, deltaZ * Time.deltaTime);
        //transform.Translate(deltaX, 0, deltaZ);
    }
}

```

В этом фрагменте кода появляется несколько новых концепций. Прежде всего, это переменная для ссылки на компонент **CharacterController**. Она просто создает локальную ссылку на объект (объект в коде — не путайте его с объектами сцены); ссылаться на этот экземпляр компонента могут разные сценарии. Изначально эта переменная пуста, поэтому, прежде чем ею пользоваться, следует указать объект, на который вы будете ссылаться с ее помощью. Именно здесь появляется метод **GetComponent()**; он возвращает другие компоненты, присоединенные к тому же объекту `GameObject`. Вместо передачи параметра в скобках мы воспользовались синтаксисом C# и определили тип в угловых скобках `<>`.

Теперь, когда у нас появилась ссылка на компонент **CharacterController**, можно вызвать для этого контроллера метод **Move()**. Мы передаем этому методу вектор тем же способом, которым код вращения в зависимости от указателя мыши использовал вектор для значений поворота. А аналогично тому, как мы ограничивали значения поворота, мы задействуем метод **Vector3.ClampMagnitude()**, чтобы ограничить модуль вектора скоростью движения; мы берем именно этот метод, потому что в противном случае движение по диагонали происходило бы быстрее движения вдоль координатных осей.

Вектор движения обладает еще одной особенностью, связанной с выбором системы координат. Вы уже сталкивались с ней при обсуждении преобразования поворота. Например, мы создадим вектор, перемещающий объект влево. Но может оказаться, что у игрока это направление не совпадает с направлением координатных осей. То есть мы говорим о локальном, а не о глобальном пространстве. Поэтому методу **Move()** следует передавать вектор движения, определенный в глобальном пространстве, а значит, нам требуется преобразование от локальных к глобальным координатам. Математически это крайне сложная операция, но, к счастью для нас, об этом позаботится Unity. Нам же достаточно вызвать метод **TransformDirection()**, чтобы, как следует из его названия, преобразовать направление.

Запустите код. Если это еще не сделано, установите для компонента **MouseLook** вращение одновременно по горизонтали и вертикали. Вы можете неограниченно осматривать пространство вокруг себя и летать, нажимая соответствующие клавиши. Это здорово, если вам требуется игрок, умеющий летать. Но как сделать, чтобы он перемещался исключительно по земле?

2.5.4. Ограничение перемещения по вертикали

Теперь, когда распознавание столкновений работает, в сценарий можно добавить силу тяжести, чтобы игрок стоял на полу. Объявим переменную **gravity** и воспользуемся ее значением для оси Y, как показано в следующем листинге.

Листинг 2.11. Добавление силы тяжести в код движения

```
public float gravity = -9.8f;
...
void Update()
{
    ...
    movement = Vector3.ClampMagnitude(movement, speed); // Ограничим движение
по диагонали той же скоростью, что и движение параллельно осям.
    movement.y = gravity; // Используем значение переменной gravity вместо нуля
    ...
}
```

Теперь на игрока действует постоянная сила, тянущая его вниз. К сожалению, она не всегда направлена строго вниз, так как изображающий игрока объект может наклоняться вниз и вверх, следуя за движениями указателя мыши. К счастью, у нас есть все, чтобы исправить этот недочет, достаточно слегка поменять настройки компонента по отношению к игроку. Первым делом ограничьте компонент **MouseLook** только горизонтальным вращением. Затем добавьте этот компонент к **камере** (она присоединена к объекту **Player**) и ограничьте его только вертикальным вращением. В результате на перемещения указателя мыши у вас будут реагировать два разных объекта.

Так как игрок теперь поворачивается только в горизонтальной плоскости, решилась проблема с наклоном вектора силы тяжести. При этом объект камеры является дочерним по отношению к игроку, поэтому, обладая способностью поворачиваться в вертикальной плоскости независимо от игрока, она вращается по горизонтали вслед за ним.

2.5.5. Совершенствование готового сценария

Воспользуйтесь методом **RequireComponent()**, чтобы проверить, присоединены ли остальные требуемые сценарию компоненты. Иногда такие компоненты являются необязательными (то есть можно сказать, что «если этот компонент присоединен, тогда...»), но бывают ситуации, когда без каких-то компонентов сценарий работать не будет. Поэтому добавьте в верхнюю часть сценария этот метод, чтобы обеспечить выполнение зависимостей, указав требуемый компонент в качестве параметра. Кроме того, можно вставить в верхнюю часть сценария метод **AddComponentMenu()**, который добавит сценарий в меню компонентов в редакторе Unity. Достаточно указать имя элемента меню, и вы получите возможность выбирать данный сценарий в

списке, открываемом щелчком на кнопке Add Component в нижней части панели Inspector. Код добавления обоих методов выглядит примерно так:

Листинг 2.12. Добавление кода проверки наличия подключенных компонент в сценарий

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[RequireComponent(typeof(CharacterController))]
[AddComponentMenu("Control Script/FPS Input")]

public class FPSInput : MonoBehaviour
```

3. Итоги

В данном разделе приводится список концепций, которые охватывает данная лабораторная работа.

1. Знакомство с составными элементами редактора Unity.
2. Создание нового проекта.
3. Создание трехмерных объектов в сцене.
4. Создание сценариев. Подключение сценария к объекту в сцене.
5. Двумерные и трехмерные координатные пространства.
6. Сцену создают помещенные в комнату объекты и источники света.
7. Коллайдеры.
8. Получение значений движения от игрока. Метод GetAxis класса Input.
9. Игрока в сцене от первого лица, по сути, представляет камера.
10. Код движения в каждом кадре циклически повторяет небольшие преобразования.
11. Элементы управления персонажем в игре от первого лица состоят из указателя мыши, отвечающего за вращение, и клавиатуры, отвечающей за перемещение.
12. Ограничение перемещений через другие объекты и добавление силы тяжести.

4. Задание по лабораторной работе

1. Ознакомьтесь с теоретическим материалом.
2. Создайте свой проект трехмерной игры в Unity.
3. В соответствии с методическими указаниями создайте прототип простого уровня.
4. Добавьте игровой объект персонажа в виде капсулы, куба или другого стандартного объекта.
5. Настройте освещение в уровне.
6. Запрограммируйте движение персонажа.
7. Настройте камеру таким образом, чтобы она реализовывала вид от 1го или от 3го лица.
8. Ответьте на контрольные вопросы.

Выполните одно из заданий по вариантам. Вариант в работе определяется в соответствии со списком группы:

1. Поменяйте объект игрока на более сложный, состоящий из нескольких элементов, таких как прототипы рук и головы. Убедитесь, что коллайдер этого объекта правильно обрабатывается игровым движком, например при взаимодействии со стенами. Добавьте наклонные поверхности на уровень. Дополните уровень полом, расположенным более высоко. Убедитесь, что объект игрока корректно движется по новому уровню.
2. Добавьте источник освещения, который бы реализовывал фонарик у игрока. Сделайте возможность включать и выключать фонарик нажатием (или зажатием) кнопки мыши.
3. Измените камеру, следящую за объектом игрока, таким образом, чтобы создать эффект покачивания при ходьбе.
4. Добавьте возможность бега при нажатии (зажатии) кнопки shift на клавиатуре.
5. Добавьте вторую камеру, которая бы реализовывала вид сверху (необязательно точно сверху), и сделайте переключение на неё по нажатию кнопки на клавиатуре.
6. Реализуйте вывод в консоль сообщений с текущими координатами игрока. Реализуйте вывод в консоль сообщений с текущей скоростью игрока.
7. Сделайте так, чтобы источники освещения в сцене периодически мигали.
8. Сделайте так, чтобы одна из стен на уровне перемещалась по горизонтали с течением времени.

5. Контрольные вопросы

1. Что такое Unity?
2. Из каких элементов состоит рабочее пространство Unity?
3. Чем отличаются окна Project и Hierarchy?
4. Какие компоненты добавляются в сцену при создании проекта?
5. Перечислите возможности интерфейса работы с панелью сцены.
6. Как добавить в сцену трехмерный объект?
7. Как создать новый сценарий? Что нужно сделать, чтобы созданный сценарий применялся при запуске игры?
8. Для чего нужен объект Camera в сцене?
9. Как вывести сообщение в консоль при запуске игры?
10. Для чего нужны методы Start() и Update()?
11. Какие вы знаете виды источников освещения? В чем разница между ними, какие они имеют основные параметры?
12. Что такое коллайдеры?

13. Опишите назначение класса Transform.

14. Опишите как осуществляется взаимодействие игровых объектов с окружением.

15. Какими методами можно задать возможность движения и вращения для игрового объекта?

16. Для чего объединять объекты в группы в панели иерархии?

17. Как реализовать независимость от частоты кадров компьютера в сценариях Unity

18. Для чего нужен компонент CharacterController?